

이해하면 인생이 바뀌는 네트워크 프로그래밍

C개발자가 Socket과 Network에 대한 본질을 이해할 수 있는 가장 쉬운 방법

넌넌한 개발자 최호성

문서 개정이력

[illegible]

수강에 앞서

수업내용 및 교육목표

- Windows 환경에서 사용할 수 있는 Wireshark 같은 패킷 분석 소프트웨어 개발
- TCP/IP 프로토콜에 대한 코드 수준의 깊은 이해
- Snort 같은 NIDS 시스템 개발에 필요한 기반 확보
- 소형 IoT 장치 개발에 필요한 네트워크 기술 이해
- 각종 네트워크 솔루션의 작동원리 이해

알고 있다고 가정하는 것들

- C언어와 선형 자료구조

- 독하게 시작하는 C 프로그래밍
- 구조체 멤버 정렬, 역호출 구조

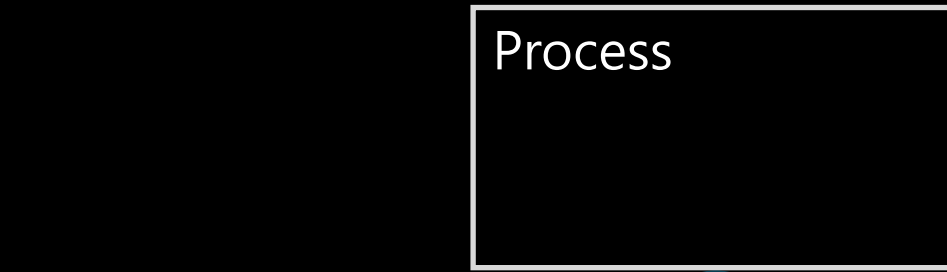
- Windows 시스템 프로그래밍 이론 일부

- 멀티스레드

- 네트워크에 대한 기본적인 이해

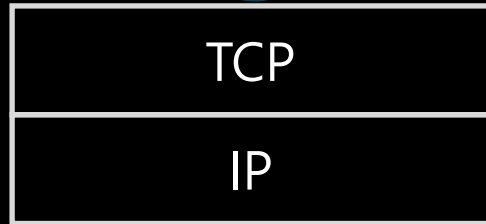
- TCP/IP 프로토콜
- 간단한 채팅 프로그램 (소켓 프로그래밍)

Socket stream, Packet, Frame



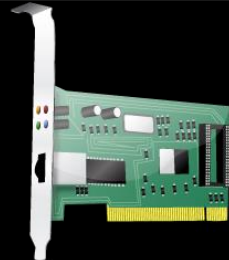
User

Kernel



S/W

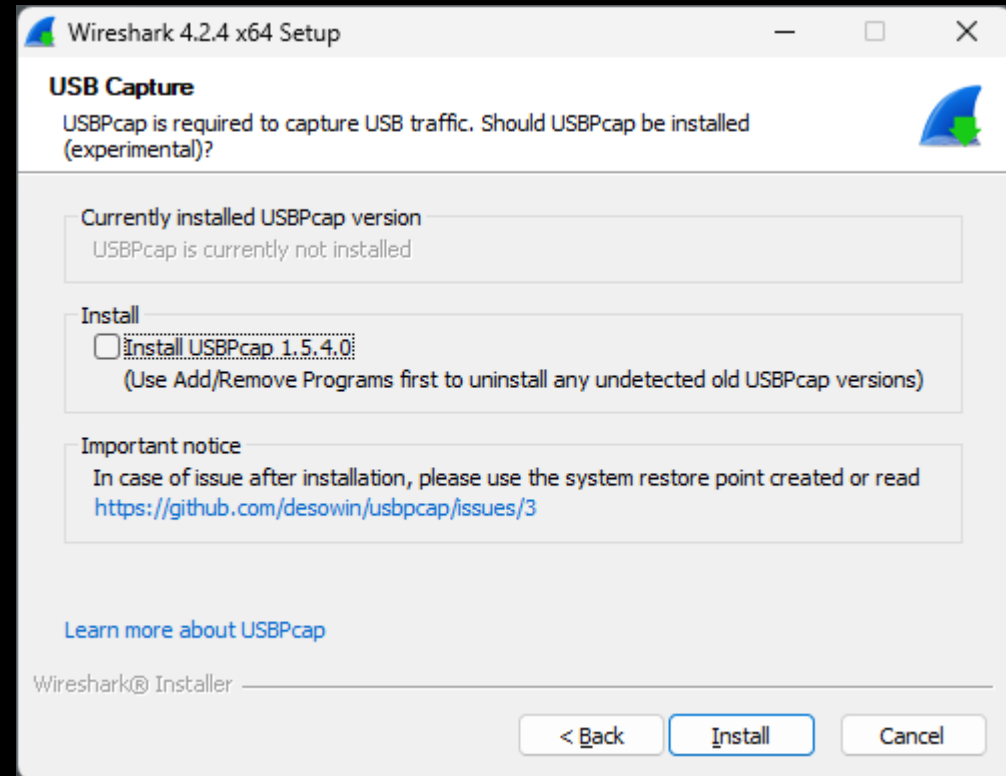
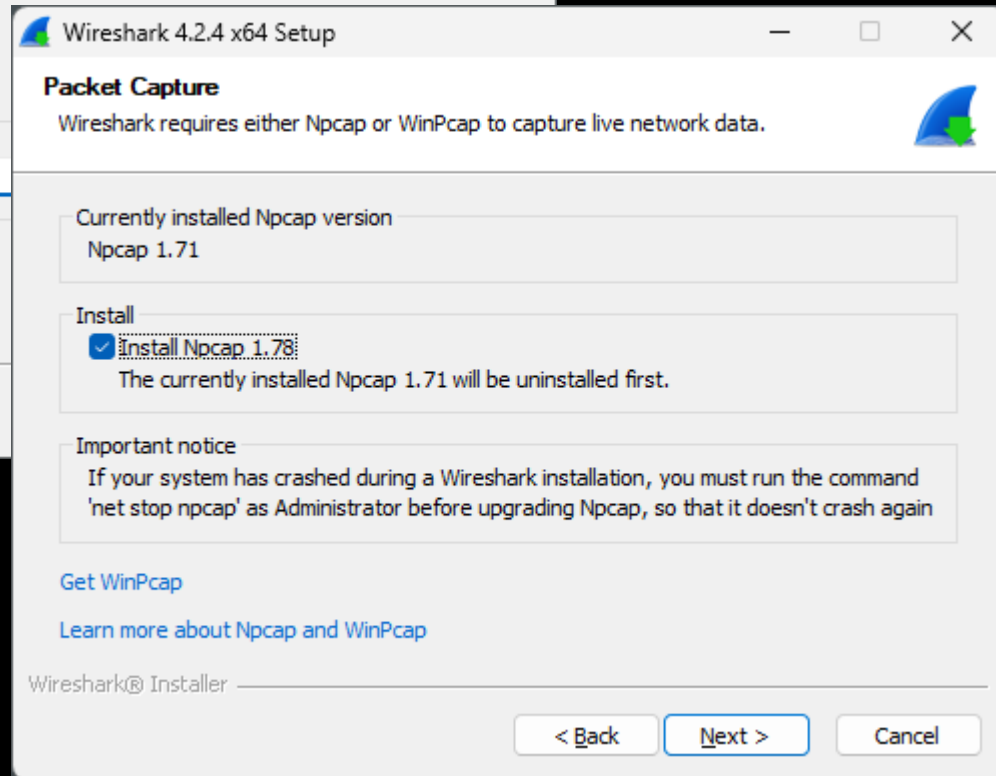
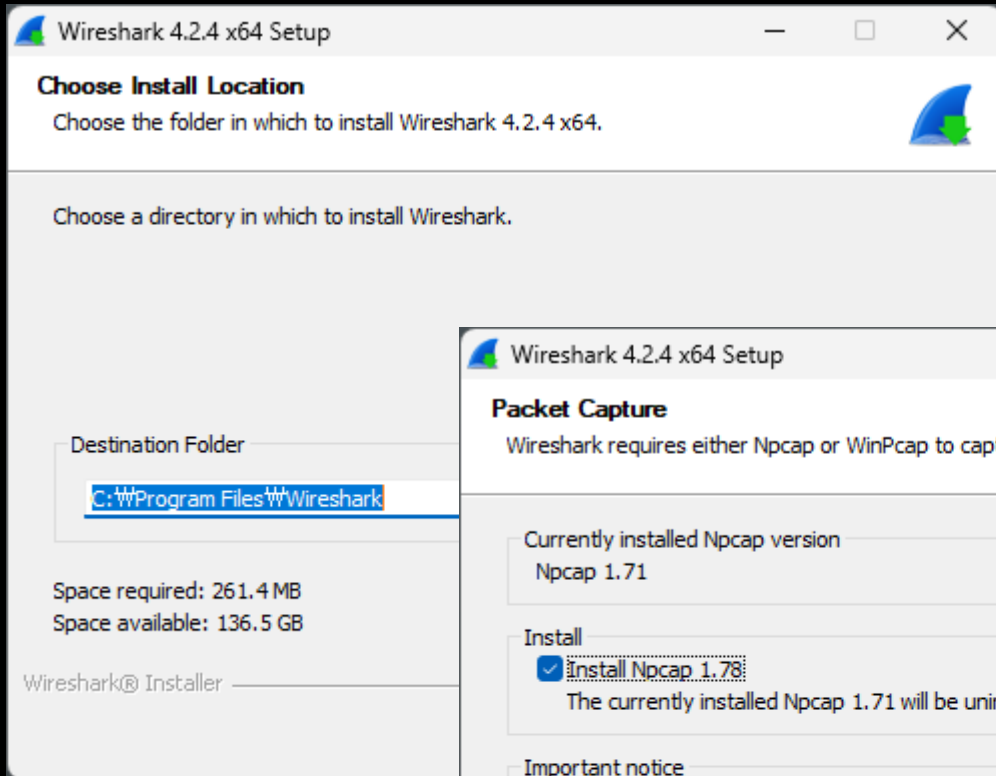
H/W



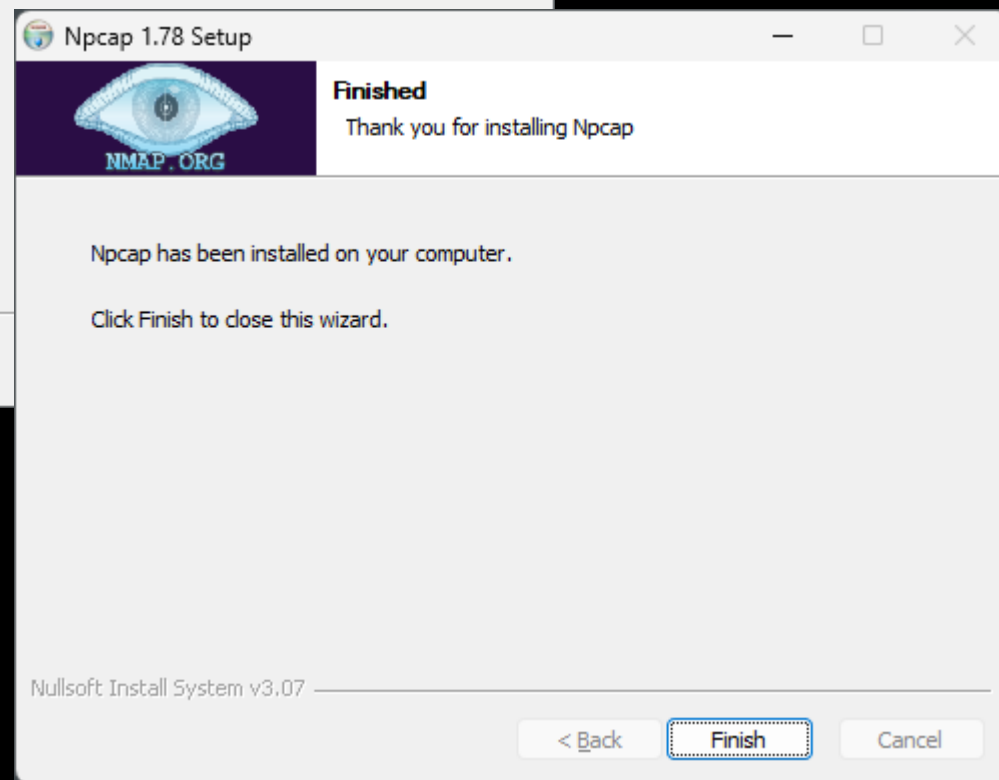
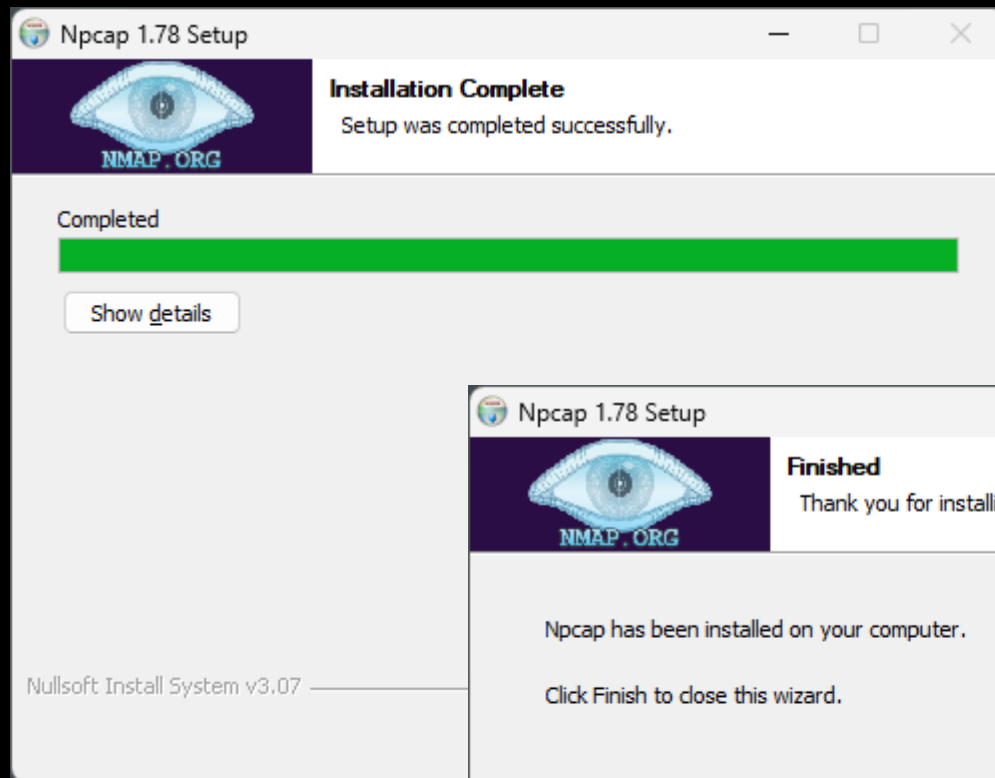
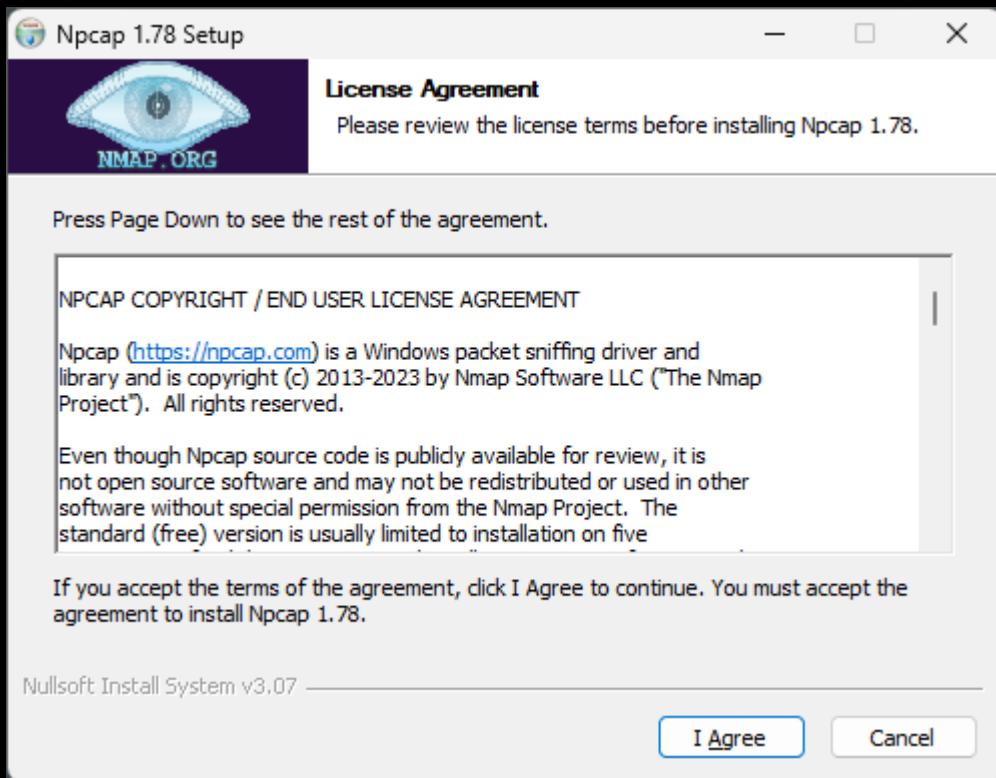
개발환경 구축

- Windows 11 (Professional)
- Visual Studio 2022 Community
- Wireshark x64 4.2.4
- Npcap SDK 1.13 (ZIP)
- VMware player 17
 - Windows 11 VM
 - 실제 환경 구성이 가능하다면 대체

Wireshark + Npcap 설치



Wireshark + Npcap 설치



Npcap 소개

<https://npcap.com/>



The screenshot shows the Npcap website with a dark blue header. In the top right corner, there are links to Nmap.org, Seclists.org, Sectools.org, and Insecure.org. The Npcap logo is on the left, followed by a search bar labeled 'Site Search'. Below the header is a navigation bar with buttons for Docs, Download, Licensing, Windows 11, and WinPcap. The main content area has a dark blue banner that reads 'Packet capture library for Windows'. Below this, a paragraph describes Npcap as the Nmap Project's packet capture and sending library for Microsoft Windows, highlighting its use of the open Pcap API and the libpcap library. It mentions that Npcap allows for sending raw packets and is compatible with Nmap and Wireshark. A bulleted list follows, detailing 'Loopback Packet Capture and Injection' capabilities, including the use of the Windows Filtering Platform (WFP) and the pcap_inject() function.

Nmap.org Seclists.org Sectools.org Insecure.org

Npcap

[Docs](#) [Download](#) [Licensing](#) [Windows 11](#) [WinPcap](#)

Packet capture library for Windows

Npcap is the Nmap Project's packet capture (and sending) library for Microsoft Windows. It implements the open [Pcap API](#) using a custom Windows kernel driver alongside our Windows build of [the excellent libpcap library](#). This allows Windows software to capture raw network traffic (including wireless networks, wired ethernet, localhost traffic, and many VPNs) using a simple, portable API. Npcap allows for sending raw packets as well. Mac and Linux systems already include the Pcap API, so Npcap allows popular software such as [Nmap](#) and [Wireshark](#) to run on all these platforms (and more) with a single codebase. Npcap began in 2013 as some improvements to the (now discontinued) WinPcap library, but has been largely rewritten since then with [hundreds of releases](#) improving Npcap's speed, portability, security, and efficiency. In particular, Npcap now offers:

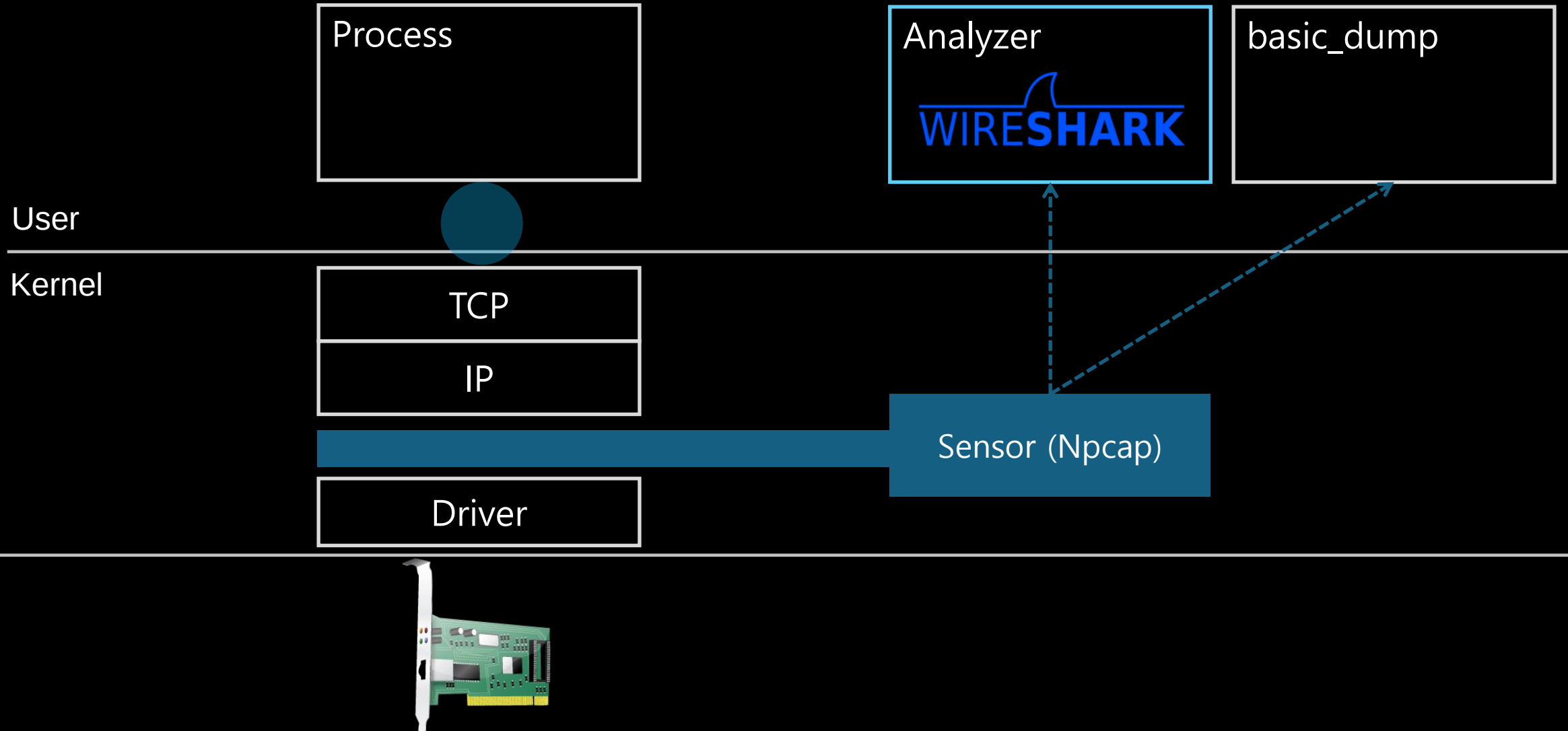
- **Loopback Packet Capture and Injection:** Npcap is able to sniff loopback packets (transmissions between services on the same machine) by using the [Windows Filtering Platform \(WFP\)](#). After installation, Npcap supplies an interface named NPF_Loopback, with the description "Adapter for loopback capture". Wireshark users can choose this adapter to capture all loopback traffic the same way as other non-loopback adapters. Packet injection works as well with the [pcap_inject\(\) function](#).

Npcap 소개

<https://npcap.com/>

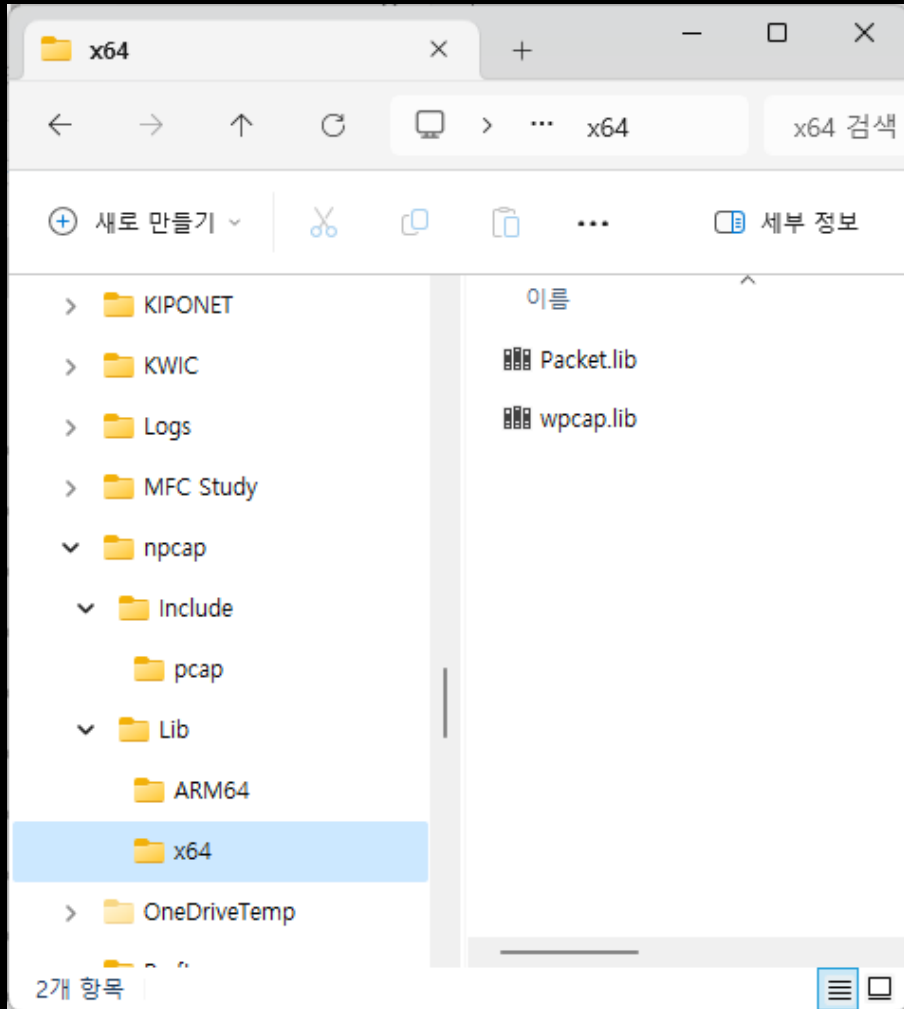
- Packet capture library for Windows
 - WinPcap에 대한 높은 호환성 제공
- 최신 버전 윈도우 지원
 - x86, x64, ARM
- Loopback 패킷 캡처 및 전송(Injection)
 - Windows Filtering Platform 활용

Npcap sensor



라이브러리 압축 해제 및 폴더 생성

C:\npcap\Include, C:\npcap\Lib\x64



- 다운로드 받은 Npcap 라이브러리를 C드라이브에 복사
- 원활한 실습을 위해 경로명 준수
- Npcap 런타임 라이브러리는 윈도우 시스템 폴더 아래 Npcap 폴더에 위치

가상 머신 구동을 위한 메인보드 설정 확인

가상 머신 전원 켜는 중...

가상 머신 이름:
WinDev2404Eval

Not in a hypervisor
partition (HVP=0)
(VERR_NEM_NOT_AVAILABL
E).

VT-x is disabled in the
BIOS for all CPU modes
(VERR_VMX_MSR_ALL_VM
X_DISABLED).

결과 코드: **E_FAIL**
(0X80004005
)

구성 요소: ConsoleWrap

인터페이스: IConsole
{6ac83d89-6e
e7-4e33-8ae6-
b257b2e81be8
}

UEFI BIOS Utility - Advanced Mode

05/02/2024 Thursday 10:54 English MyFavorite(F3) Qfan Control(F6) EZ Tuning Wizard(F11)

My Favorites Main Ai Tweaker **Advanced** Monitor Boot Tool Exit

L3 Cache12 MB

L4 CacheN/A

Intel VT-x TechnologySupported

Intel SMX TechnologySupported

Software Guard Extensions (SGX)

Software Controlled

Tcc Offset Time Window

Auto

Hardware Prefetcher

Enabled

Adjacent Cache Line Prefetch

Enabled

Intel (VMX) Virtualization Technology

Enabled

Maximum CPU Core Temperature

Auto

Active Processor Cores

All

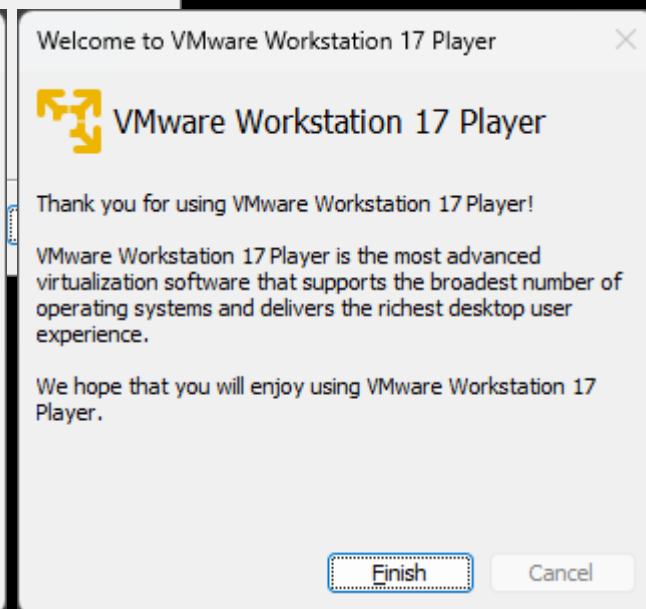
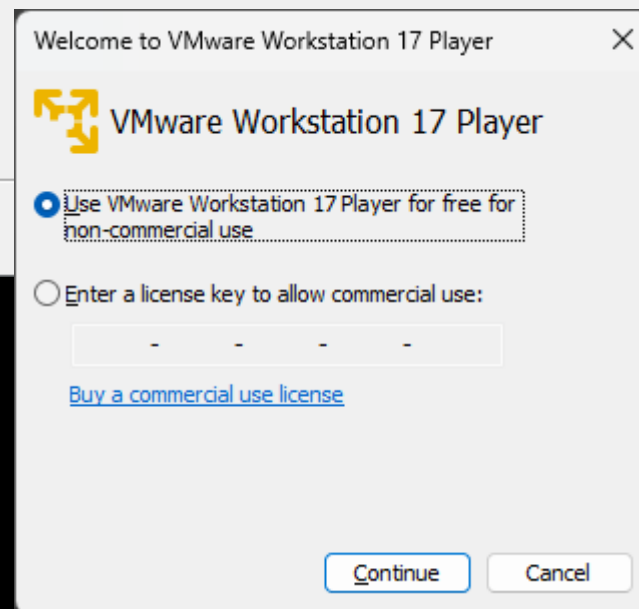
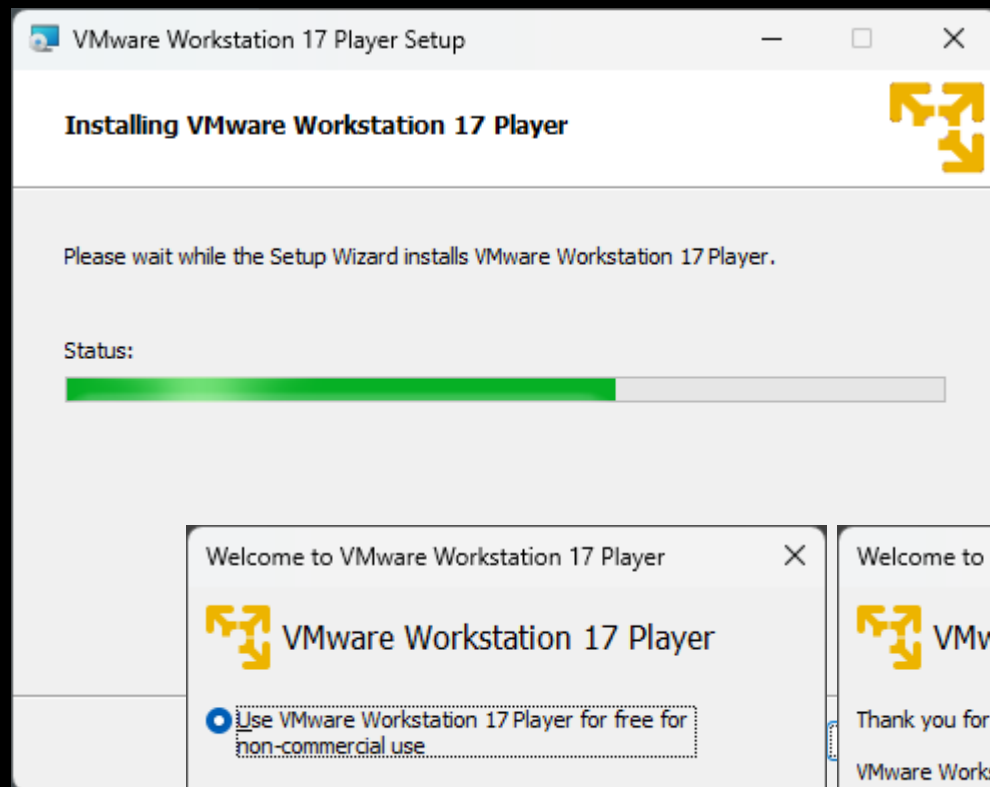
Thermal Monitor

Enabled

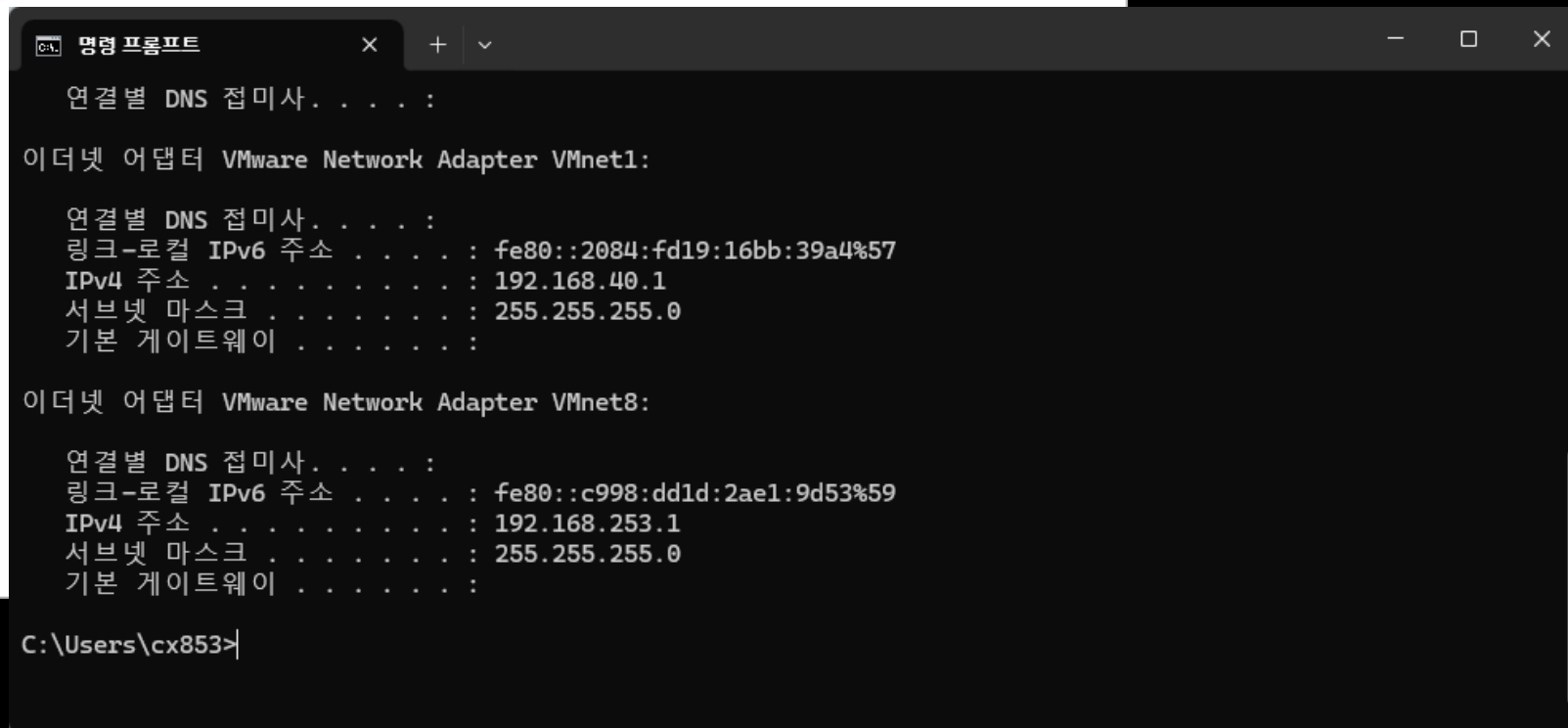
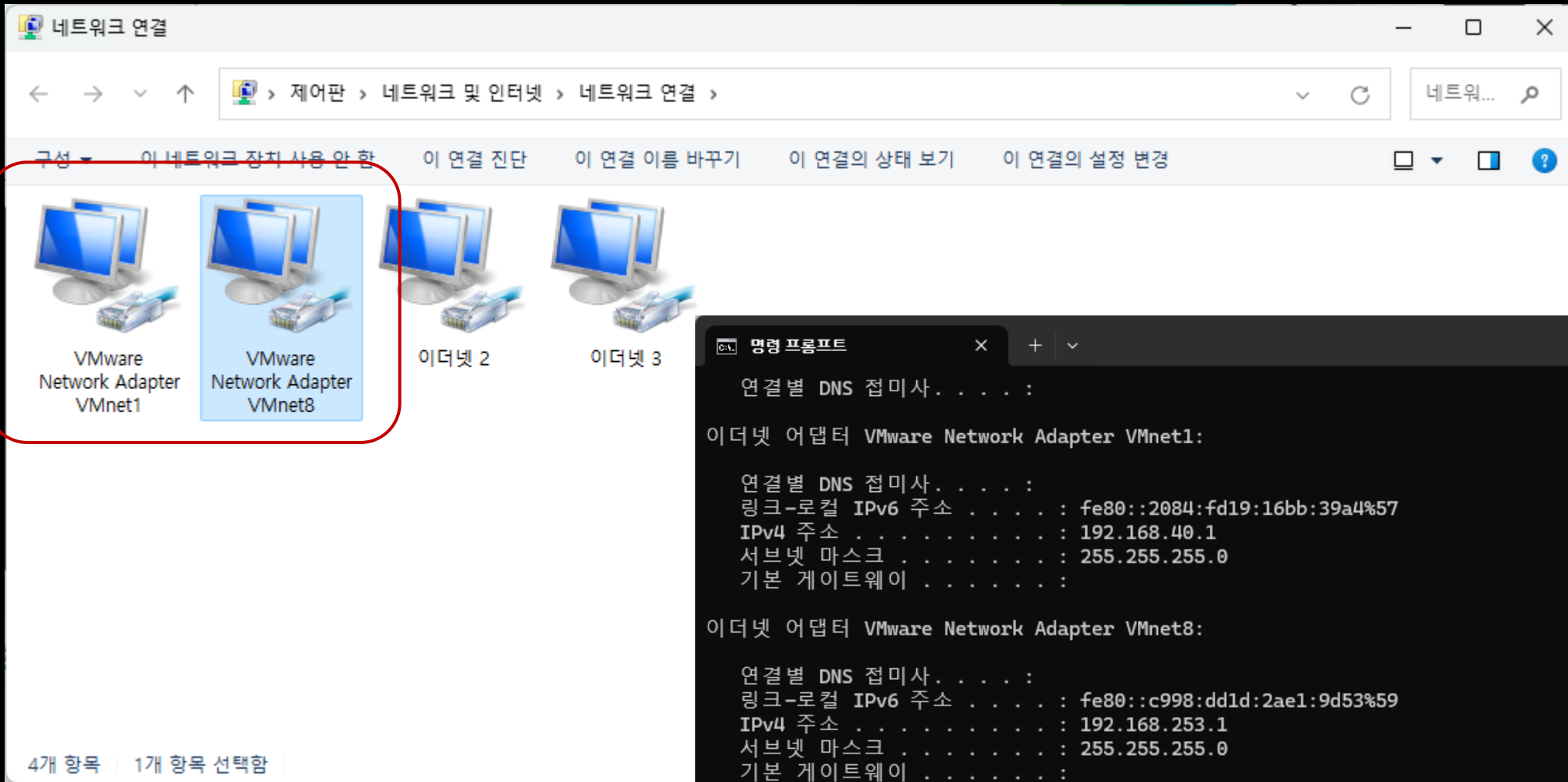
> CPU - Power Management Control

When enabled, a VMM can utilize the additional hardware capabilities provided by Vanderpool Technology.

VMware player 17 설치

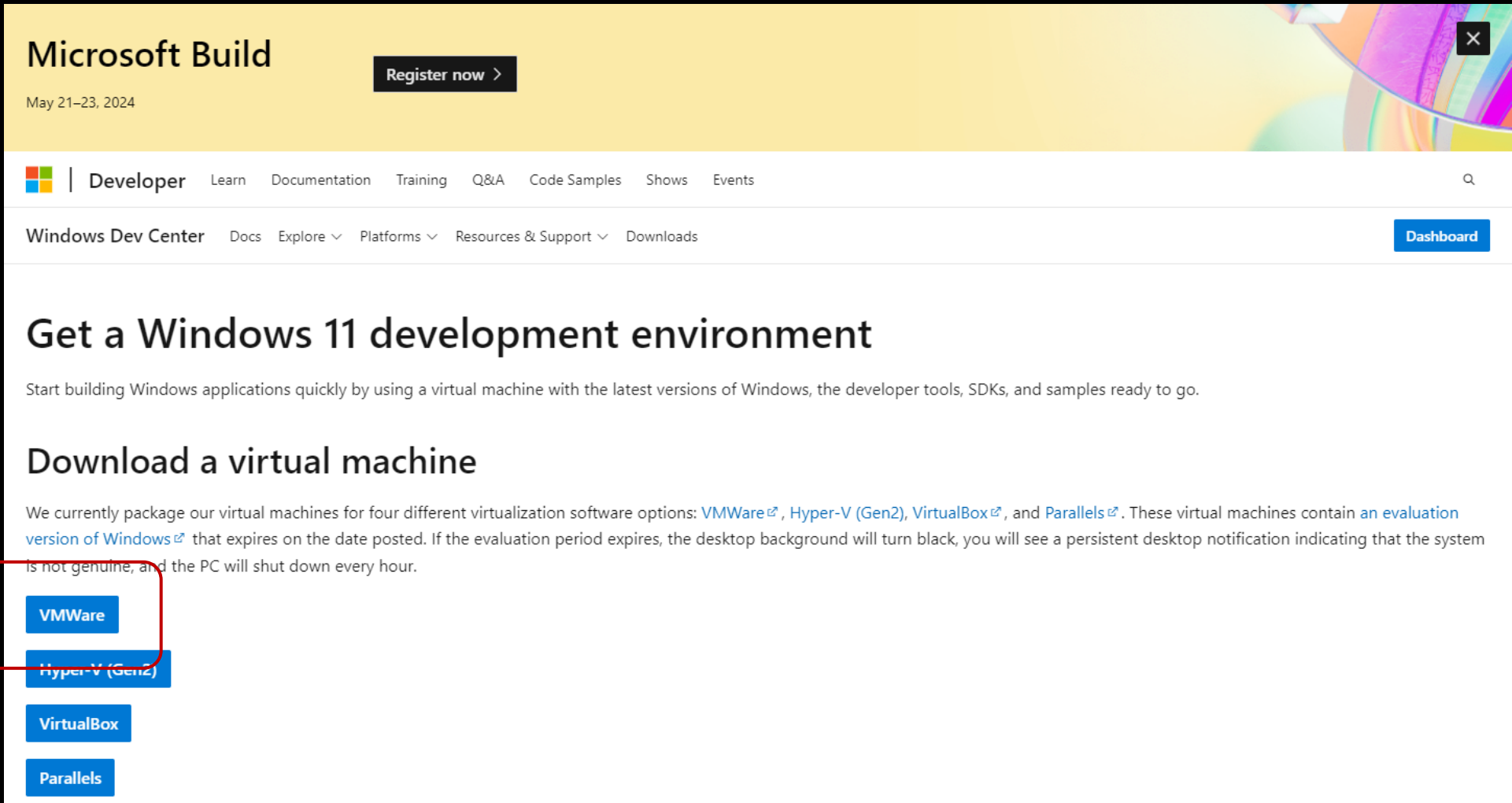


가상 어댑터 설치 확인



Windows 11 가상머신 다운로드

<https://developer.microsoft.com/en-us/windows/downloads/virtual-machines/>



The screenshot shows the Microsoft Build website. At the top, there's a yellow banner for "Microsoft Build" with the dates "May 21-23, 2024" and a "Register now" button. Below the banner is a navigation bar with links for "Developer", "Learn", "Documentation", "Training", "Q&A", "Code Samples", "Shows", and "Events". A search icon is on the right. Below the navigation bar is a "Windows Dev Center" section with links for "Docs", "Explore", "Platforms", "Resources & Support", and "Downloads". A "Dashboard" button is on the right. The main content area has the heading "Get a Windows 11 development environment" and a subheading "Download a virtual machine". Below this, there's a paragraph explaining that virtual machines are available for four different virtualization software options: VMWare, Hyper-V (Gen2), VirtualBox, and Parallels. A red box highlights the "Hyper-V (Gen2)" button.

Microsoft Build
May 21-23, 2024
[Register now](#)

Developer | [Learn](#) [Documentation](#) [Training](#) [Q&A](#) [Code Samples](#) [Shows](#) [Events](#)

Windows Dev Center [Docs](#) [Explore](#) [Platforms](#) [Resources & Support](#) [Downloads](#) [Dashboard](#)

Get a Windows 11 development environment

Start building Windows applications quickly by using a virtual machine with the latest versions of Windows, the developer tools, SDKs, and samples ready to go.

Download a virtual machine

We currently package our virtual machines for four different virtualization software options: [VMWare](#), [Hyper-V \(Gen2\)](#), [VirtualBox](#), and [Parallels](#). These virtual machines contain [an evaluation version of Windows](#) that expires on the date posted. If the evaluation period expires, the desktop background will turn black, you will see a persistent desktop notification indicating that the system is not genuine, and the PC will shut down every hour.

[VMWare](#)
[Hyper-V \(Gen2\)](#)
[VirtualBox](#)
[Parallels](#)

Windows 11 VM 구동

VMware Workstation 17 Player (Non-commercial use only)

Player ▾ | ▶ ▾ | 🖨️ | 🖼️ | 🗑️

내 PC > DEV (F:) > VM > WinDev2404Eval.VMWare

정렬 ▾ | 보기 ▾ | ...

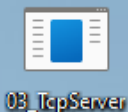
Welcome to VMware Workstation 17 Player

-  **Create a New Virtual Machine**
Create a new virtual machine, which will then be added to the of your library.
-  **Open a Virtual Machine**
Open an existing virtual machine, which will then be added to top of your library.
-  **Upgrade to VMware Workstation Pro**
Get advanced features such as snapshots, virtual network management, and more.
-  **Help**
View online help.
-  This product is not licensed and is authorized for non-commercial use only. For commercial use, purchase a license. [Buy now.](#)

WinDev2404Eval.mf

WinDev2404Eval.ovf

WinDev2404Eval-disk1.vmdk



Windows 11 Enterprise Evaluation
Windows License valid for 89 days
Build 22621.ni_release.220506-1250

```
Command Prompt
C:\Users\User>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet0:

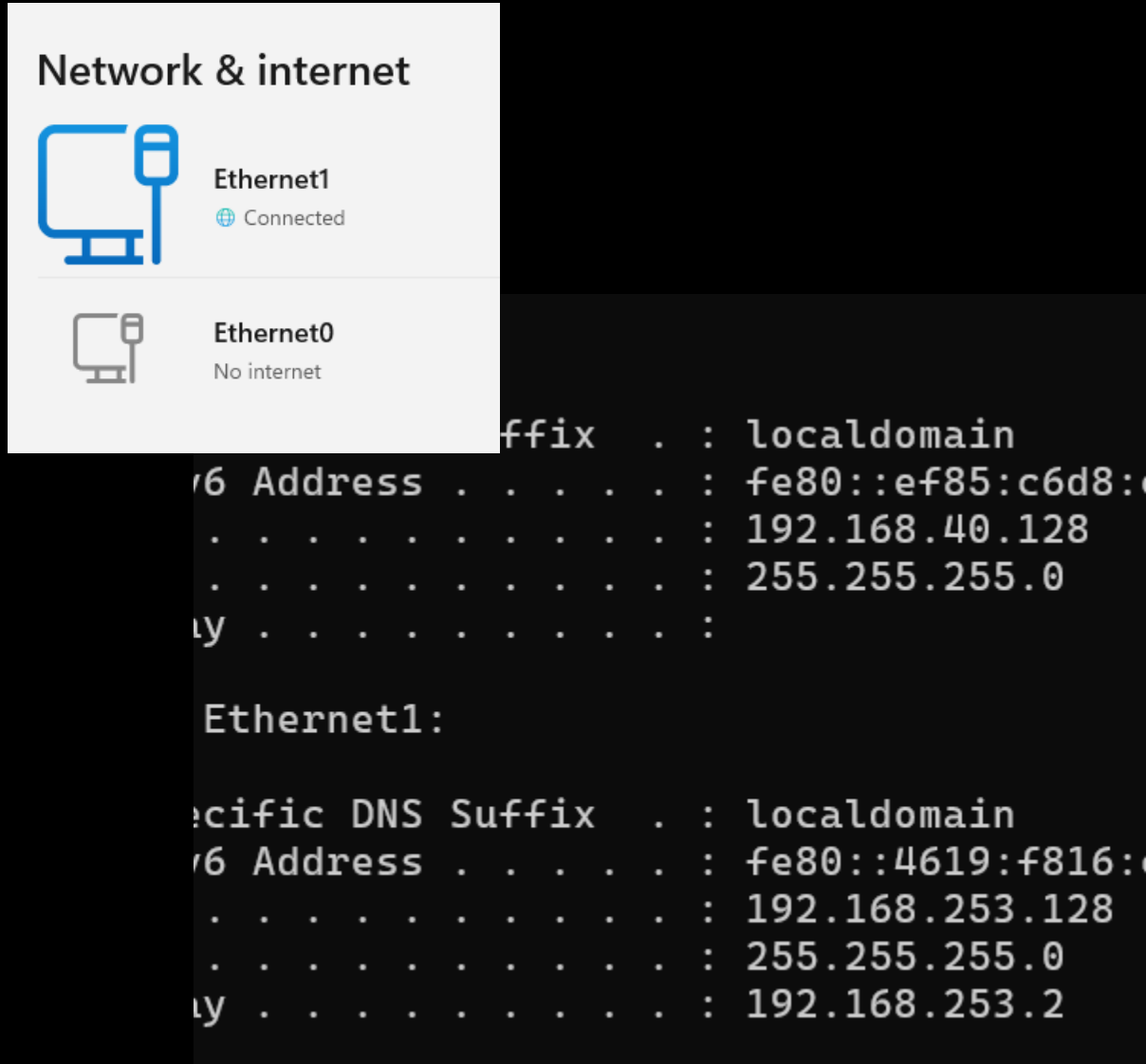
    Connection-specific DNS Suffix  . : localdomain
    Link-local IPv6 Address . . . . . : fe80::ef85:c6d8:ea9c:5802%10
    IPv4 Address. . . . . : 192.168.40.128
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

Ethernet adapter Ethernet1:

    Connection-specific DNS Suffix  . : localdomain
    Link-local IPv6 Address . . . . . : fe80::4619:f816:c4e0:12fe%11
    IPv4 Address. . . . . : 192.168.253.128
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.253.2

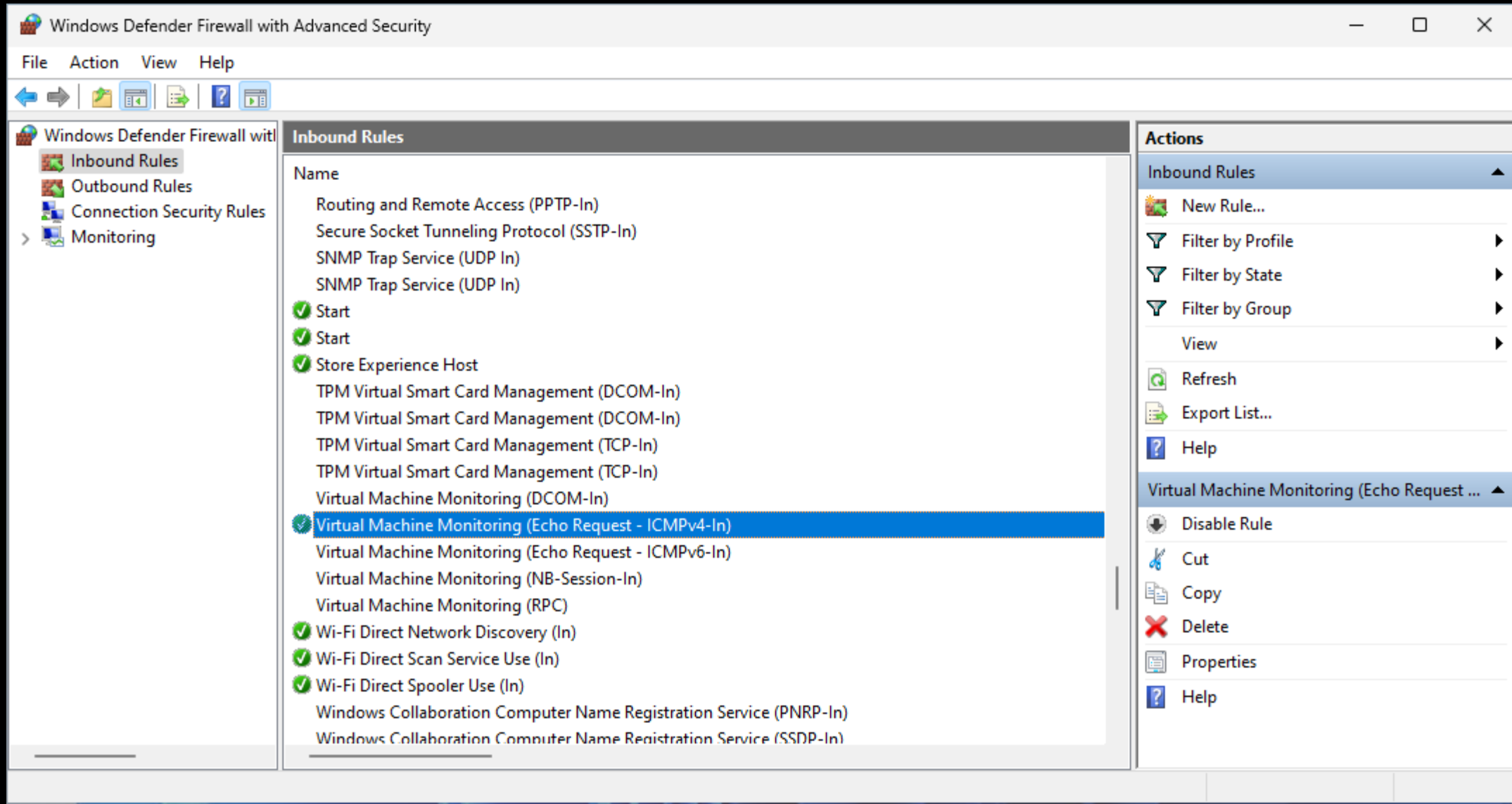
Ethernet adapter vEthernet (WSL):
```

Guest OS 환경설정

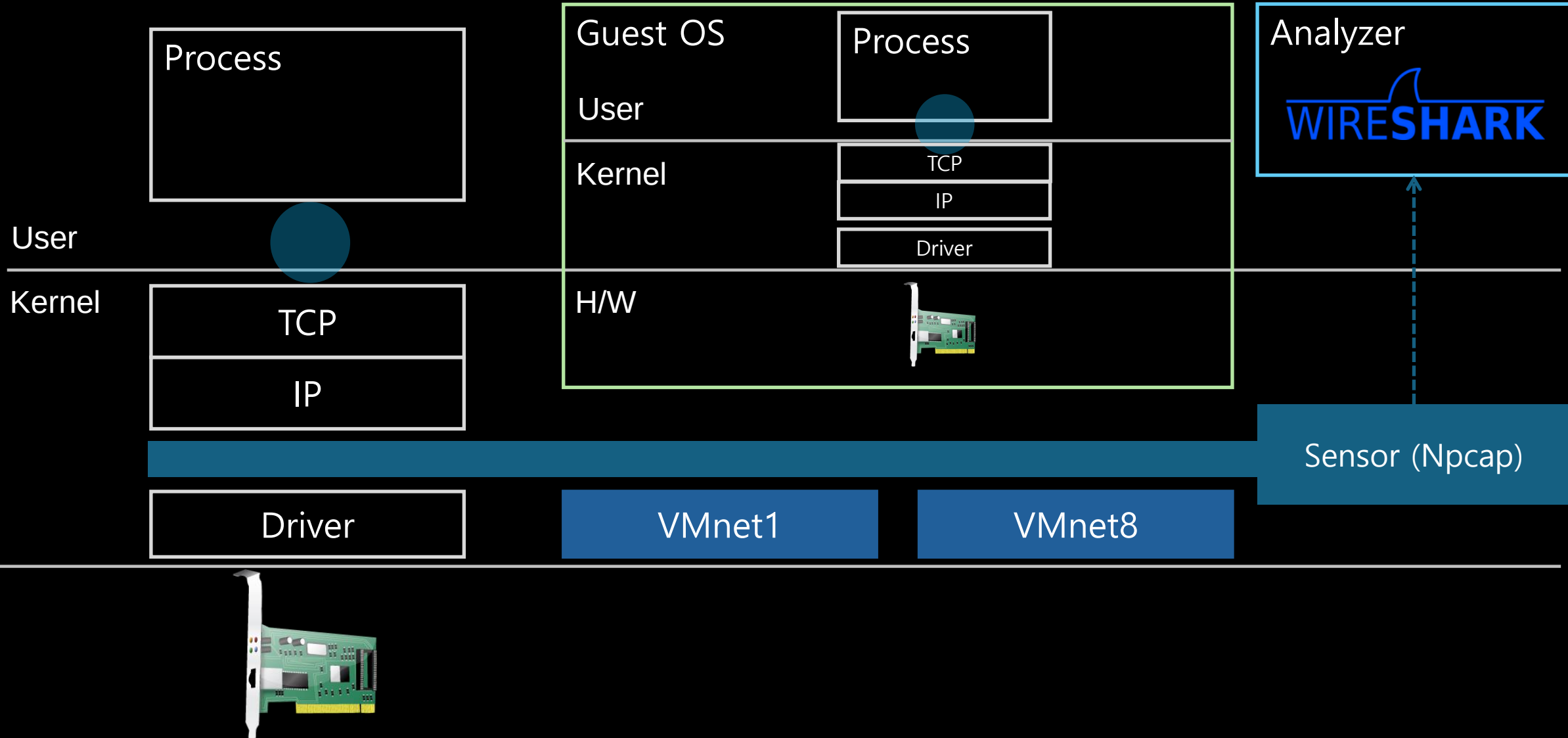


- Windows 11 VM은 2개 NIC을 사용
- Ethernet0
 - Host OS와 통신을 위해 사용하는 인터페이스
- Ethernet1
 - VM 자체가 인터넷에 접근하기 위한 인터페이스
 - NAT

Guest OS ping 확인을 위한 설정변경 (선택)



VMware player 네트워크 작동 구조



Host OS CPU 점유율 증가 문제

VMware Network Adapter VMnet8에서 캡처 중 (응답 없음)

파일(F) 편집(E) 보기(V) 이동(G) 캡처(C) 분석(A) 통계(S) 전화(Y) 무선(W) 도구(T) 도움말(H)

표시 필터 적용 ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Host
3919	25.930767	192.168.253.2	192.168.253.128	DNS	181	
3920	25.934277	192.168.253.128	192.168.253.2	DNS	78	
3921	25.960209	192.168.253.128	192.168.253.2	ICMP	74	
3922	25.960227	192.168.253.2	192.168.253.128	ICMP	74	
3923	25.966301					
3924	26.286021					
3925	26.584241					
3926	27.286351					
3927	28.086501					
3928	28.287031					
3929	29.214501					
3930	29.344991					
3931	29.345391					
3932	29.346101					
3933	29.346131					
3934	29.477001					

작업 관리자

프로세스

성능

앱 기록

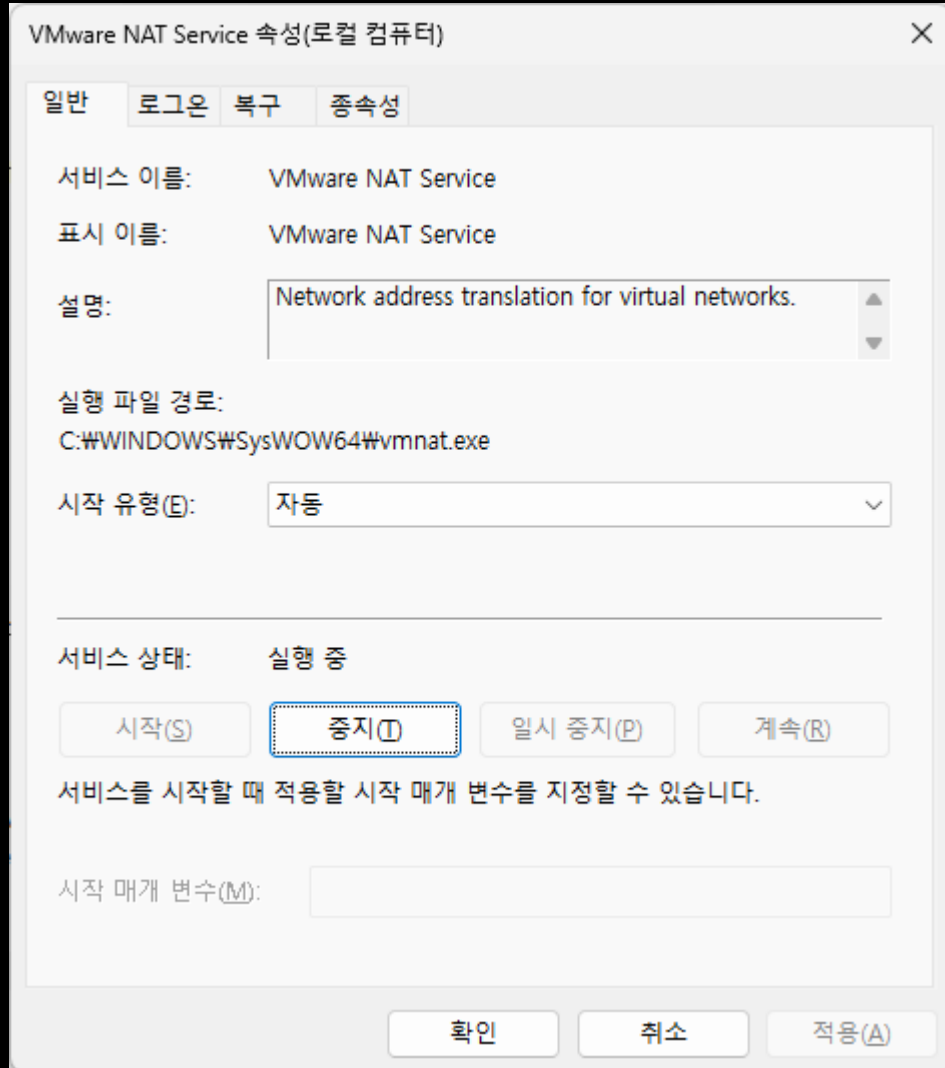
시작 앱

사용자

검색할 이름, 게시자 또는 PID를 입력하세요.

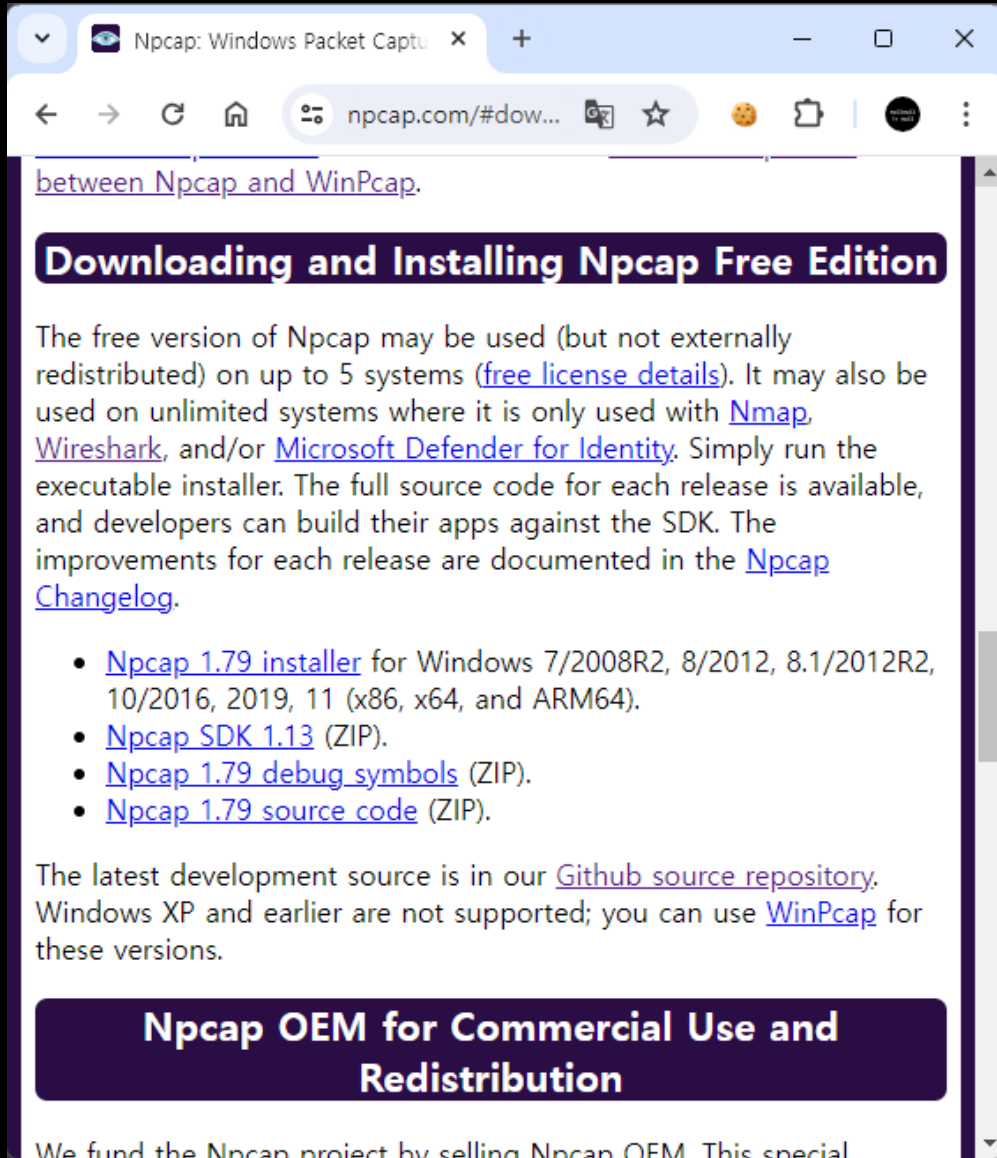
이름	상태	61% CPU	27% 메모리	2% 디스크	4% 네트워크
> VMware NAT Service(32비트)		25.1%	2.3MB	0MB/s	9.5Mbps
> Wireshark		20.4%	2,465.8MB	0MB/s	0Mbps
VMware Workstation VMX		9.1%	32.4MB	14.8MB/s	0Mbps
> Initech Client Manager Service		0.8%	2.1MB	0MB/s	0Mbps
데스크톱 창 관리자		0.8%	59.9MB	0MB/s	0Mbps

NAT Service 재시작



- Host OS NIC 중 VMnet1 인터페이스는 Guest OS와 통신하기 위한 인터페이스
- VMnet8 인터페이스는 공유기로 기능
- 두 가상 어댑터에 대해 패킷 모니터링 시 CPU 점유율 증가
 - VM에 Wireshark를 설치해 사용하는 방식으로 대응

Npcap SDK 설치 및 예제 빌드



- Npcap SDK 1.13 (ZIP)
 - 압축해제 후 수동으로 설치
 - SDK에 샘플 예제가 포함되어 있음
- 런타임 라이브러리는 Wireshark 설치 시 이미 설치하므로 별도로 설치 할 필요 없음

Visual Studio 프로젝트 설정

C:\npcap\Include, C:\npcap\Lib\x64

▼ 공용 프로젝트 콘텐츠

공용 포함 디렉터리	
모든 헤더 파일이 공용임	아니요
공용 C++ 모듈 디렉터리	
모든 모듈이 공용임	아니요

▼ 일반

실행 가능 디렉터리	\$(VC_ExecutablePath_x64);\$(C
포함 디렉터리	\$(VC_IncludePath);\$(WindowsS
외부 include 디렉터리	ude;C:\npcap\Include\pcap;▼
참조 디렉터리	\$(VC_ReferencesPath_x64);
라이브러리 디렉터리	\$(VC_LibraryPath_x64);\$(Winc
라이브러리 WinRT 디렉터리	\$(WindowsSDK_MetadataPath);
소스 디렉터리	\$(VC_SourcePath);

외부 include 디렉터리

컴파일하는 동안 외부/시스템으로 처리할 경로이며, 빌드 최신 검사...

▼ 공용 프로젝트 콘텐츠

공용 포함 디렉터리	
모든 헤더 파일이 공용임	아니요
공용 C++ 모듈 디렉터리	
모든 모듈이 공용임	아니요

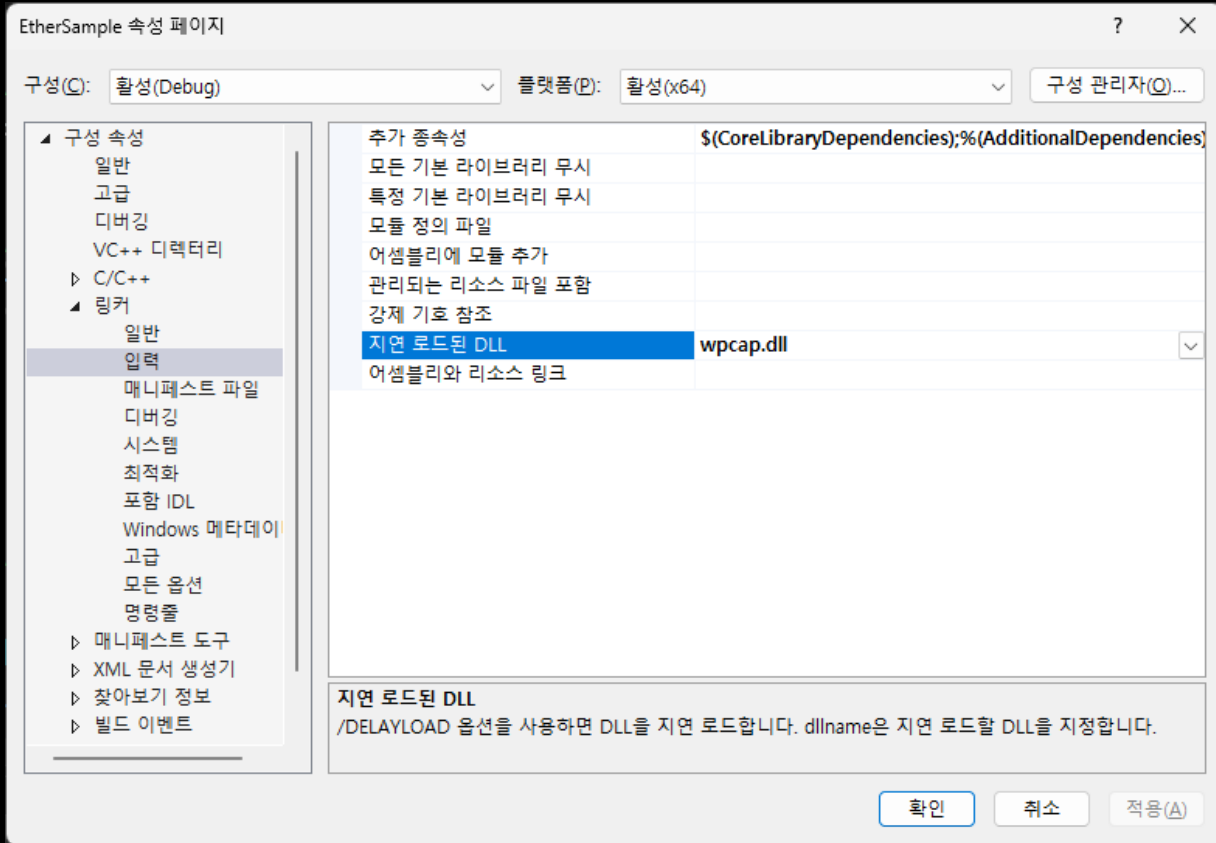
▼ 일반

실행 가능 디렉터리	\$(VC_ExecutablePath_x64);\$(C
포함 디렉터리	\$(VC_IncludePath);\$(WindowsS
외부 include 디렉터리	\$(VC_IncludePath);\$(WindowsS
참조 디렉터리	\$(VC_ReferencesPath_x64);
라이브러리 디렉터리	ath_x64);C:\npcap\Lib\x64;▼
라이브러리 WinRT 디렉터리	\$(WindowsSDK_MetadataPath);
소스 디렉터리	\$(VC_SourcePath);

라이브러리 디렉터리

VC++ 프로젝트를 빌드하는 동안 라이브러리 파일을 검색할 때 사용...

Visual Studio 프로젝트 설정



- Npcap 런타임 라이브러리는 반드시 지연 로드 하도록 설정
 - wpcap.dll
- 지연 로드를 적용 할 경우 프로그램 시작 시 즉시 DLL을 로드하는 것이 아니라 호출 시점으로 늦춤
 - wpcap.dll 로드 오류 방지

Trace 파일 다운로드

<https://www.chappell-university.com/>

CHAPPELL UNIVERSITY

Training

Certification

Customers

Books

Laura's Lab Blog

More

TCP/IP Network Analysis and Forensics

Expand your knowledge and
take your career to the next level.

Subscribe to the Packet Advisor Newsletter

Get the latest info on Wireshark, network analysis, and network forensics right to your inbox.

Email Address

First Name

Last Name

You can unsubscribe at any time by clicking the link in our newsletter. By clicking subscribe, you agree with our [privacy policy](#) and acknowledge

Trace 파일 다운로드

<https://www.chappell-university.com/traces>
(C:\SampleTraces)

CHAPPELL UNIVERSITY

[Training](#)[Certification](#)[Customers](#)[Books](#)[Laura's Lab Blog](#)[More](#)

Student Portal

Welcome to the Chappell University Student Portal. We have hundreds of trace files to help our students and instructors.

Download Instructions:

Public Google Folder (Updated March 2021)

Click [here](#) to access the files on our public Google folder.

FTP or Browser Access (see warning below)

Site: wiresharktraces.com

Username: traces@wiresharktraces.com

Password: wireshark2020

Warning: In March 2021, a Chrome update disabled "Enable support for FTP URLs" - if you try using Chrome with this setting - take a trace file! You'll see you won't even send a DNS request to resolve the name. This is a clear sign that your system is blocking access to a target. We've now copied all the trace files over to a Google folder (see above). We think disabling FTP URL support is a good security move, but it caught us/others off guard. Apologies if you hit this issue.

Questions/access problems? Contact info@chappellU.com and we'll try to assist a.s.a.p.



http-browse-ok
.pcap



http-redirects.p
cap



ip-checksum-in
valid.pcap



ip-fragments.pc
ap



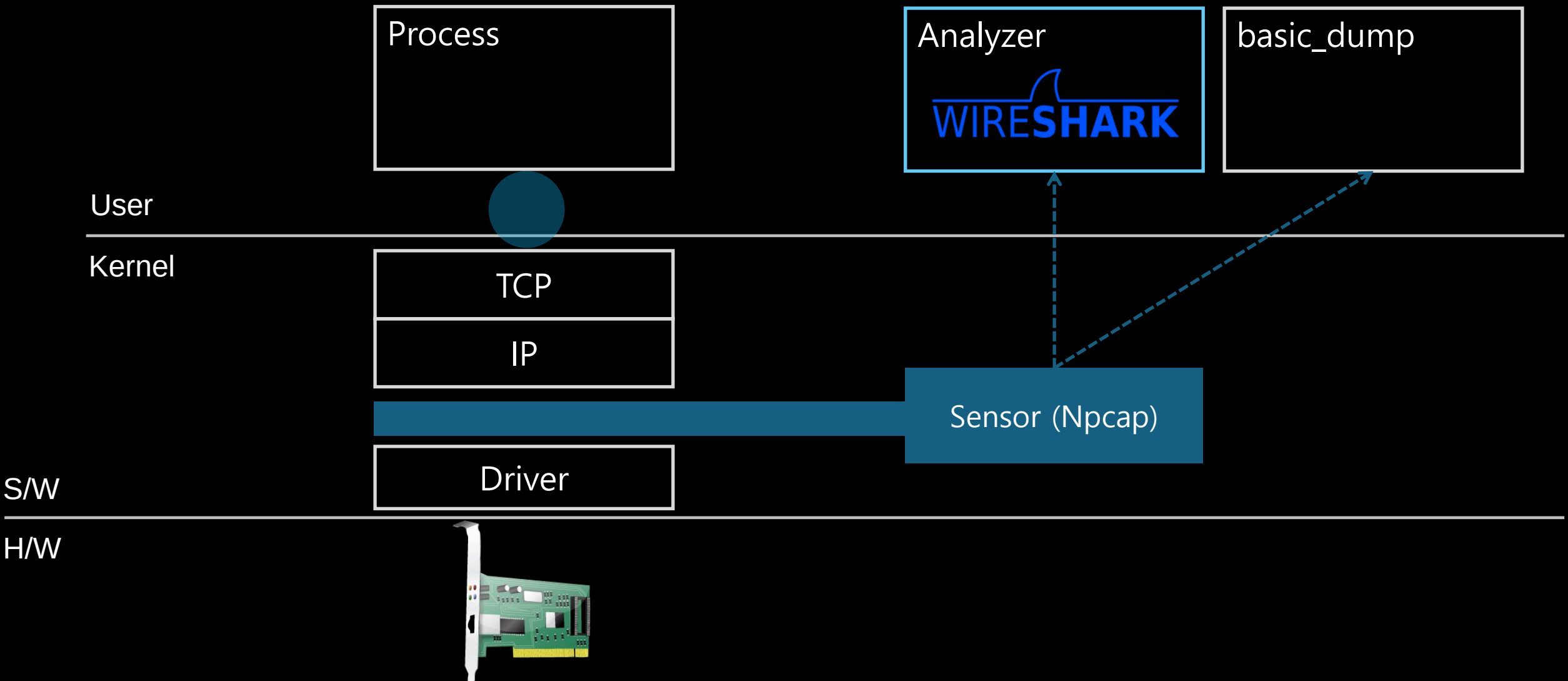
udp-echo.pcap

샘플 예제 분석

basic_dump 예제 실행 화면

```
F:\Npcap으로 패킷 분석기 작 × + ▾  
1. \Device\NPF_{3BA0EEAB-694A-4805-A7CE-FCDA37220C54} (Intel(R) Ethernet Connection (7) I219-V)  
2. \Device\NPF_{75CD8E73-599F-49D0-8ABB-176CD8AF12D1} (VMware Virtual Ethernet Adapter)  
3. \Device\NPF_{B795FF90-8B10-4783-8A5F-2AF9EAA98BA6} (VMware Virtual Ethernet Adapter)  
Enter the interface number (1-3):1  
  
listening on Intel(R) Ethernet Connection (7) I219-V...  
10:46:31,092817 len:501  
10:46:31,092839 len:66  
10:46:31,252057 len:66  
10:46:31,590153 len:1347  
10:46:31,636517 len:54  
10:46:31,713804 len:42  
|
```

예제 작동 구조



주요 함수 소개

- `pcap_findalldevs()`
 - `pcap_freealldevs()` 함수로 해제
- `pcap_open_live()`
 - `pcap_close()` 함수로 종료
- `pcap_loop()`
 - `packet_handler()` 함수 Callback
- `pcap_freealldevs()`
- `pcap_close()`

basic_dump 예제 분석

```
int main()
{
    pcap_if_t *alldevs;
    pcap_if_t *d;
    int inum;
    int i=0;
    pcap_t *adhandle;
    char errbuf[PCAP_ERRBUF_SIZE];

#ifdef _WIN32
    /* Load Npcap and its functions. */
    if (!LoadNpcapDlls())
    {
        fprintf(stderr, "Couldn't load Npcap\n");
    }
}
```

basic_dump 예제 분석

```
/* Retrieve the device list */
if(pcap_findalldevs(&alldevs, errbuf) == -1)
{
    fprintf(stderr, "Error in pcap_findalldevs: %s\n", errbuf);
    exit(1);
}

/* Print the list */
for(d=alldevs; d; d=d->next)
{
    printf("%d. %s", ++i, d->name);
    if (d->description)
        printf(" (%s)\n", d->description);
    else
        printf(" (No description available)\n");
}
```

basic_dump 예제 분석

```
printf("Enter the interface number (1-%d):", i);  
scanf("%d", &inum);  
  
if(inum < 1 || inum > i)  
{  
    printf("\nInterface number out of range.\n");  
    /* Free the device list */  
    pcap_freealldevs(alldevs);  
    return -1;  
}  
  
/* Jump to the selected adapter */  
for(d=alldevs, i=0; i< inum-1 ;d=d->next, i++);
```

basic_dump 예제 분석

```
/* Open the device */
/* Open the adapter */
if ((adhandle= pcap_open_live(d->name,      // name of the device
                              65536,        // portion of the packet to
                              |             // 65536 grants that the whc
                              1,             // promiscuous mode (nonzero
                              1000,         // read timeout
                              errbuf        // error buffer
                              )) == NULL)
{
    fprintf(stderr, "\nUnable to open the adapter. %s is not supported\n", d->name);
    /* Free the device list */
    pcap_freealldevs(alldevs);
    return -1;
}
```

basic_dump 예제 분석

```
printf("\nlistening on %s...\n", d->description);

/* At this point, we don't need any more the device list.
pcap_freealldevs(alldevs);

/* start the capture */
pcap_loop(adhandle, 0, packet_handler, NULL);

pcap_close(adhandle);
return 0;
```

basic_dump 예제 분석

```
/* Callback function invoked by libpcap for every incoming packet */
void packet_handler(u_char *param,
    const struct pcap_pkthdr *header, const u_char *pkt_data)
{
    struct tm *ltime;
    char timestr[16];
    time_t local_tv_sec;

    /*
     * unused parameters
     */
    (VOID)(param);
    (VOID)(pkt_data);
}
```

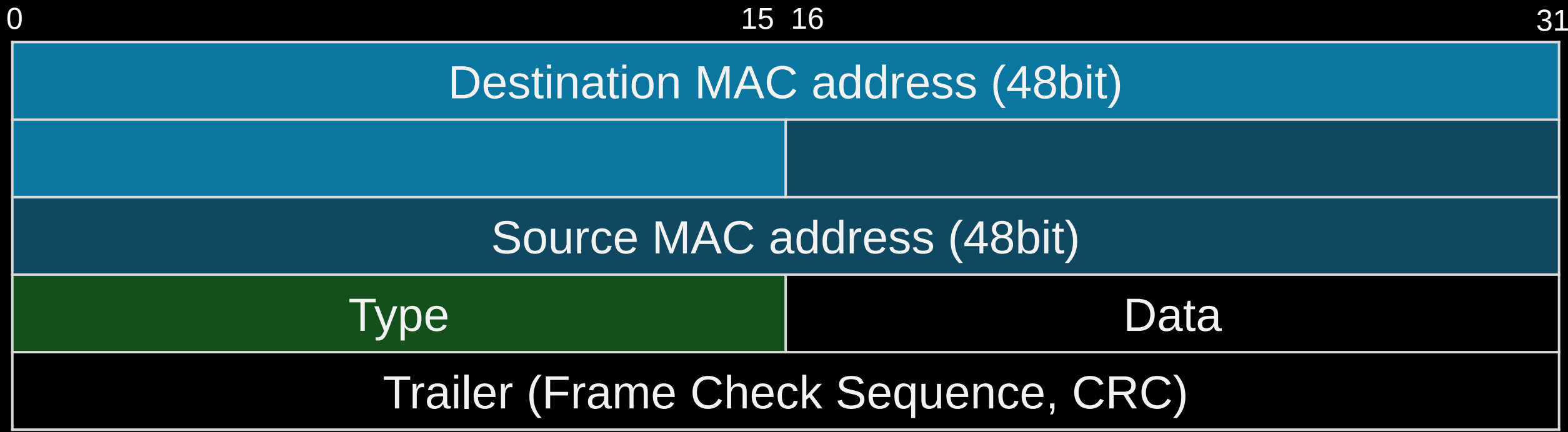
basic_dump 예제 분석

```
/*
 * unused parameters
 */
(VOID)(param);
(VOID)(pkt_data);

/* convert the timestamp to readable format */
local_tv_sec = header->ts.tv_sec;
ltime=localtime(&local_tv_sec);
strftime( timestr, sizeof timestr, "%H:%M:%S", ltime);

printf("%s,%.6d len:%d\n", timestr, header->ts.tv_usec, header->len);
```


Ethernet 헤더



Type 값



Internet Assigned Numbers Authority

Ethertype (decimal)	Ethertype (hex)	Exp. Ethernet (decimal)	Exp. Ethernet (octal)	Description	Reference
0000-1500	0000-05DC	-	-	IEEE802.3 Length Field	[IEEE Std 802.3]
0257-0511	0101-01FF	-	-	Old Xerox Experimental values. Invalid as an EtherType since 1983.	[Neil Sembower]
2048	0800	513	1001	Internet Protocol version 4 (IPv4)	[RFC9542]
2049	0801	-	-	X.75 Internet	[Neil Sembower]
2050	0802	-	-	NBS Internet	[Neil Sembower]
2051	0803	-	-	ECMA Internet	[Neil Sembower]
2052	0804	-	-	Chaosnet	[Neil Sembower]
2053	0805	-	-	X.25 Level 3	[Neil Sembower]
2054	0806	-	-	Address Resolution Protocol (ARP)	[RFC9542]

Ethernet 헤더 분석 및 출력 – 헤더 정의

```
#pragma pack(push, 1)
typedef struct EtherHeader {
    unsigned char dstMac[6];
    unsigned char srcMac[6];
    unsigned short type;
} EtherHeader;
#pragma pack(pop)
```

Ethernet 헤더 분석 및 출력

```
EtherHeader* pEther = (EtherHeader*)pkt_data;

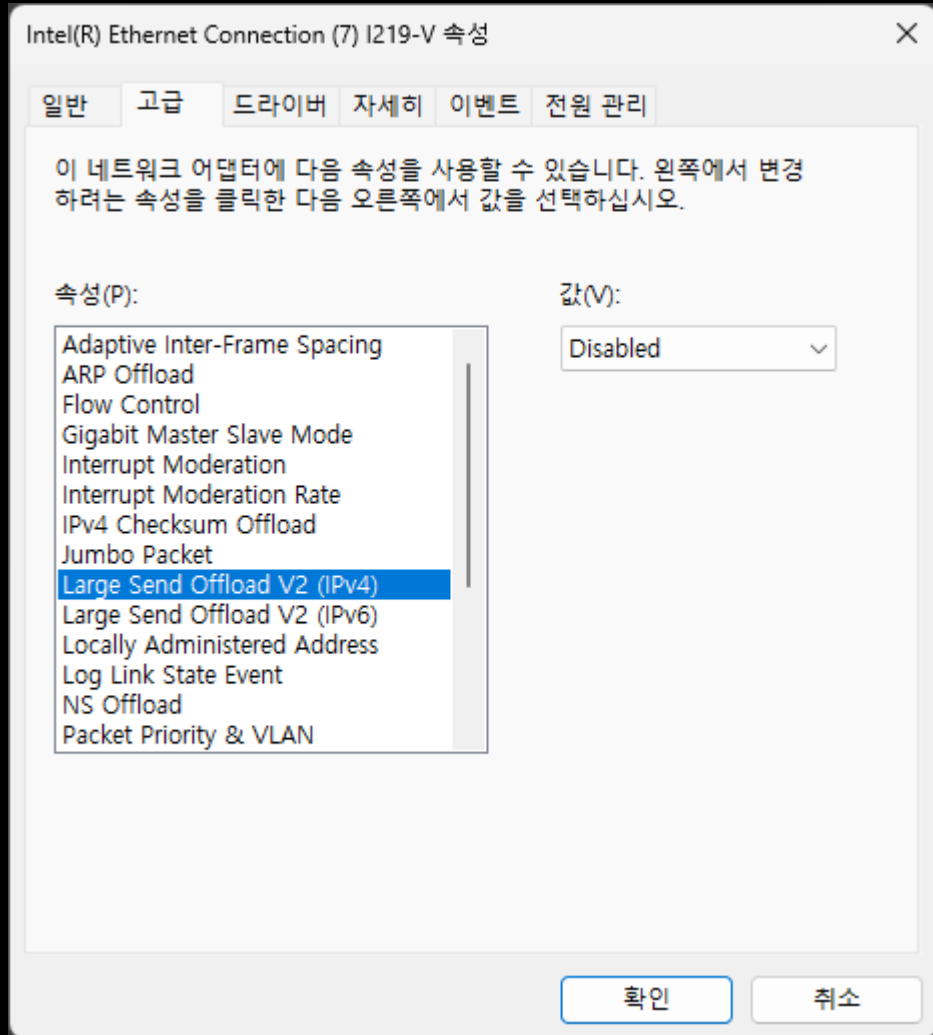
printf("%s,%.6d len:%d, "
      "SRC: %02X-%02X-%02X-%02X-%02X-%02X -> "
      "DST: %02X-%02X-%02X-%02X-%02X-%02X, type:%04X\n",
      timestr, header->ts.tv_usec, header->len,
      pEther->srcMac[0], pEther->srcMac[1], pEther->srcMac[2],
      pEther->srcMac[3], pEther->srcMac[4], pEther->srcMac[5],
      pEther->dstMac[0], pEther->dstMac[1], pEther->dstMac[2],
      pEther->dstMac[3], pEther->dstMac[4], pEther->dstMac[5],
      htons(pEther->type));
```

L2 Frame 데이터 출력

```
F:\Npcap으로 패킷 분석기 작 × + ▾
C:\WINDOWS\system32\Npcap
1. \Device\NPF_{3BA0EEAB-694A-4805-A7CE-FCDA37220C54} (Intel(R) Ethernet Connection (7) I219-V)
2. \Device\NPF_{75CD8E73-599F-49D0-8ABB-176CD8AF12D1} (VMware Virtual Ethernet Adapter)
3. \Device\NPF_{B795FF90-8B10-4783-8A5F-2AF9EAA98BA6} (VMware Virtual Ethernet Adapter)
Enter the interface number (1-3):1

listening on Intel(R) Ethernet Connection (7) I219-V...
10:41:33,619702 len:1514, SRC: 0C-9D-92-02-B9-80 -> DST: 04-D4-C4-25-15-F1, type:0800
10:41:33,619803 len:1514, SRC: 0C-9D-92-02-B9-80 -> DST: 04-D4-C4-25-15-F1, type:0800
10:41:33,712026 len:1514, SRC: 0C-9D-92-02-B9-80 -> DST: 04-D4-C4-25-15-F1, type:0800
10:41:33,712065 len:1514, SRC: 0C-9D-92-02-B9-80 -> DST: 04-D4-C4-25-15-F1, type:0800
10:41:35,118143 len:1953, SRC: 04-D4-C4-25-15-F1 -> DST: 0C-9D-92-02-B9-80, type:0800
10:41:35,123455 len:1514, SRC: 0C-9D-92-02-B9-80 -> DST: 04-D4-C4-25-15-F1, type:0800
10:41:35,123456 len:1514, SRC: 0C-9D-92-02-B9-80 -> DST: 04-D4-C4-25-15-F1, type:0800
10:41:35,123457 len:1514, SRC: 0C-9D-92-02-B9-80 -> DST: 04-D4-C4-25-15-F1, type:0800
10:41:35,488363 len:1514, SRC: 0C-9D-92-02-B9-80 -> DST: 04-D4-C4-25-15-F1, type:0800
10:41:35,488364 len:1514, SRC: 0C-9D-92-02-B9-80 -> DST: 04-D4-C4-25-15-F1, type:0800
10:41:35,488365 len:1514, SRC: 0C-9D-92-02-B9-80 -> DST: 04-D4-C4-25-15-F1, type:0800
10:41:42,756664 len:3932, SRC: 04-D4-C4-25-15-F1 -> DST: 0C-9D-92-02-B9-80, type:0800
10:41:53,750492 len:3250, SRC: 04-D4-C4-25-15-F1 -> DST: 0C-9D-92-02-B9-80, type:0800
|
```

TCP Segmentation offload



- 스트림 데이터를 TCP 세그먼트로 분할 시 NIC이 수행하도록 설정하는 것 (통상 기본값은 사용)
- 1Gbps급 NIC에서는 큰 문제가 없으나 10Gbps급 고속 환경에서 장애이슈가 있을 수 있음

Pcap 파일 형식 및 읽기

01_PcapFileSample

```
/* Open the capture file */
if ((fp = pcap_open_offline(
    "C:\\SampleTraces\\ip-fragments.pcap",
    errbuf)) == NULL)
{
    fprintf(stderr, "\nUnable to open the file %s.\n",
        "C:\\SampleTraces\\ip-fragments.pcap");
    return -1;
}

/* read and dispatch packets until EOF is reached */
pcap_loop(fp, 0, dispatcher_handler, NULL);
```

IP 헤더 분석

IP 헤더 형식

0		15		16		31	
Version		IHL		ToS		Total length	
Identification				0	D F	M F	Fragment offset
TTL		Protocol		Checksum			
Source address							
Destination address							
Options (0~40)							
Payload (Data)							

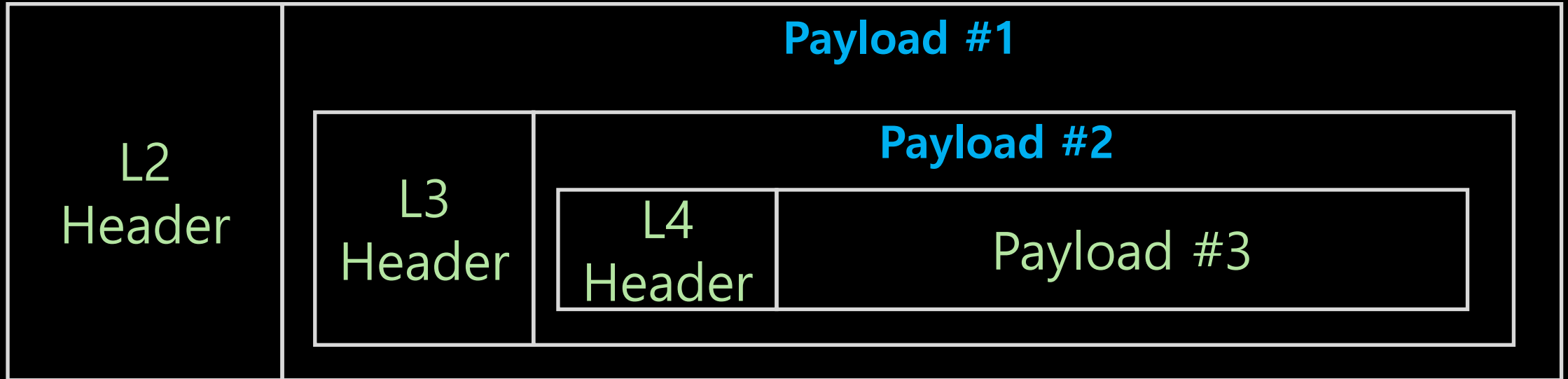
0		15		16		31	
Version		IHL		ToS		Total length	
Identification				0	D F	M F	Fragment offset
TTL		Protocol		Checksum			
Source address							
Destination address							
Options (0~40)							
Payload (Data)							

IP 헤더 구조체 정의

```
typedef struct IpHeader {  
    unsigned char verIhl;  
    unsigned char tos;  
    unsigned short length;  
    unsigned short id;  
    unsigned short fragOffset;  
    unsigned char ttl;  
    unsigned char protocol;  
    unsigned short checksum;  
    unsigned char srcIp[4];  
    unsigned char dstIp[4];  
} IpHeader;
```

Encapsulation

상위계층 헤더 및 데이터가 캡슐화 되는 것!



수작업 패킷 분석

이 Frame 데이터가 캡슐화 하고 있는 것은?

```
00 04 5a df 80 c7 00 d0 59 0b b9 44 08 00 45 00
00 30 01 c0 40 00 80 06 4b c5 c0 a8 01 b6 a1 3a
49 aa 04 40 00 50 ce e4 2f a4 00 00 00 00 70 02
40 00 92 c3 00 00 02 04 05 b4 01 01 04 02
```

정답 및 Wireshark 분석 결과

```
> Frame 1: 62 bytes on wire (496 bits), 62 bytes captured (496 bits)
~ Ethernet II, Src: AmbitMicrosy_0b:b9:44 (00:d0:59:0b:b9:44), Dst: LinksysGroup_df:80:c7 (00:04:5a:df:80:c7)
  > Destination: LinksysGroup_df:80:c7 (00:04:5a:df:80:c7)
  > Source: AmbitMicrosy_0b:b9:44 (00:d0:59:0b:b9:44)
  Type: IPv4 (0x0800)
~ Internet Protocol Version 4, Src: 192.168.1.182, Dst: 161.58.73.170
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 48
  Identification: 0x01c0 (448)
  > 010. .... = Flags: 0x2, Don't fragment
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 128
  Protocol: TCP (6)
  Header Checksum: 0x4bc5 [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 192.168.1.182
  Destination Address: 161.58.73.170
```

IP 버전, IHL, Total length

0		15		16		31	
Version		IHL		ToS		Total length	
Identification				0	D F	M F	Fragment offset
TTL		Protocol		Checksum			
Source address							
Destination address							
Options (0~40)							
Payload (Data)							

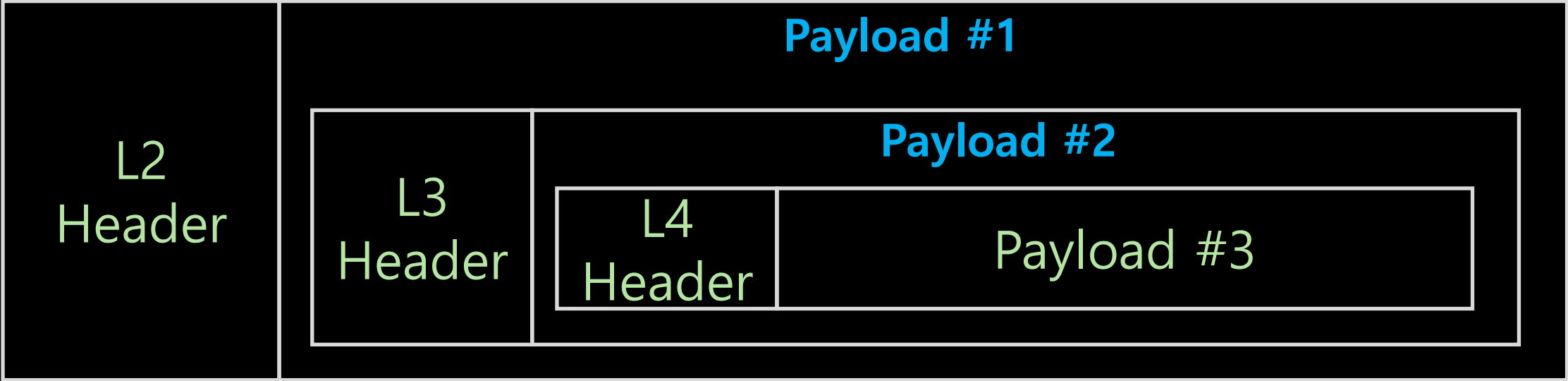
IP 버전, IHL, Total length

- 2024년 현재 사용되는 IP 프로토콜 버전은 4, 6
 - 본 수업에서는 IPv4에 대해서만 다룸
- IHL(Internet Header Length)
 - IP 헤더의 길이
 - 32비트(4바이트) 단위
- Total length
 - IP 패킷의 바이트 단위 길이
 - 헤더 길이 포함
 - Network order 주의

IP 헤더에서 버전, IHL 추출

```
EtherHeader* pEther = (EtherHeader*)pkt_data;
if (pEther->type != 0x0008)
    return;

IpHeader* pIpHeader = (IpHeader*)(pkt_data + sizeof(EtherHeader));
printf("IPv%d, IHL: %d, Total length: %d\n",
       (pIpHeader->verIhl & 0xF0) >> 4,
       (pIpHeader->verIhl & 0x0F) * 4,
       ntohs(pIpHeader->length));
```

TTL, Protocol

0		15		16		31	
Version		IHL		ToS		Total length	
Identification				0	D F	M F	Fragment offset
TTL		Protocol		Checksum			
Source address							
Destination address							
Options (0~40)							
Payload (Data)							

0		15		16		31	
Version		IHL		ToS		Total length	
Identification				0	D F	M F	Fragment offset
TTL		Protocol		Checksum			
Source address							
Destination address							
Options (0~40)							
Payload (Data)							

TTL, Protocol

- TTL(Time To Live)

- 한 Hop을 지날 때마다 1씩 감소하며 0이 되면 목적지 전달에 실패한 것으로 간주하고 폐기

- Protocol

- Payload에 담긴 정보를 해석하기 위한 상위 계층 프로토콜 번호

IANA 프로토콜 번호

Decimal ⌵	Keyword ⌵	Protocol ⌵	IPv6 Extension Header ⌵	Reference ⌵
0	HOPOPT	IPv6 Hop-by-Hop Option	Y	[RFC8200]
1	ICMP	Internet Control Message		[RFC792]
2	IGMP	Internet Group Management		[RFC1112]
3	GGP	Gateway-to- Gateway		[RFC823]
4	IPv4	IPv4 encapsulation		[RFC2003]
5	ST	Stream		[RFC1190] [RFC1819]
6	TCP	Transmission Control		[RFC9293]
7	CBT	CBT		[Tony Ballardie]
8	EGP	Exterior Gateway Protocol		[RFC888] [David Mills]
17	UDP	User Datagram		[RFC768] [Jon Postel]

출발지/목적지 IP주소

0		15		16		31	
Version		IHL		ToS		Total length	
Identification				0	D F	M F	Fragment offset
TTL		Protocol		Checksum			
Source address							
Destination address							
Options (0~40)							
Payload (Data)							

출발지/목적지 IP주소

192.168.0.10

1100 0000	1010 1000	0000 0000	0000 1010
1111 1111	1111 1111	1111 1111	0000 0000
1100 0000	1010 1000	0000 0000	0000 0000

TTL, Protocol, IP주소 추출

```
printf("TTL: %d, Protocol: %02X, Checksum: %04X\n",  
    pIpHeader->ttl,  
    pIpHeader->protocol,  
    ntohs(pIpHeader->checksum));  
  
printf("%d.%d.%d.%d -> %d.%d.%d.%d\n",  
    pIpHeader->srcIp[0], pIpHeader->srcIp[1],  
    pIpHeader->srcIp[2], pIpHeader->srcIp[3],  
    pIpHeader->dstIp[0], pIpHeader->dstIp[1],  
    pIpHeader->dstIp[2], pIpHeader->dstIp[3]  
);
```

02_IpSample 실행결과

F:\Npcap으로 패킷 분석기 작

+ v

- □ ×

```
1. \Device\NPF_{8443A134-778D-4CB7-BE5D-B5ED95DCBCEF} (WAN Miniport (Network Monitor))
2. \Device\NPF_{EB295B87-F0E2-4666-8E90-0B07A5E39CE9} (WAN Miniport (IPv6))
3. \Device\NPF_{B50665C3-9648-40BA-9AEE-DBF26D528A5D} (WAN Miniport (IP))
4. \Device\NPF_{75CD8E73-599F-49D0-8ABB-176CD8AF12D1} (VMware Virtual Ethernet Adapter
for VMnet8)
5. \Device\NPF_{B795FF90-8B10-4783-8A5F-2AF9EAA98BA6} (VMware Virtual Ethernet Adapter
for VMnet1)
6. \Device\NPF_{3BA0EEAB-694A-4805-A7CE-FCDA37220C54} (Intel(R) Ethernet Connection (7)
I219-V)
7. \Device\NPF_Loopback (Adapter for loopback traffic capture)
Enter the interface number (1-7):6
```

```
listening on Intel(R) Ethernet Connection (7) I219-V...
```

```
IPv4, IHL: 20, Total length: 73
```

```
TTL: 108, Protocol: 06, Checksum: ECA5
```

```
52.108.46.20 -> 192.168.50.42
```

```
IPv4, IHL: 20, Total length: 40
```

```
TTL: 128, Protocol: 06, Checksum: 0000
```

```
192.168.50.42 -> 52.108.46.20
```

```
IPv4, IHL: 20, Total length: 79
```


0		15		16		31	
Version	IHL	ToS		Total length			
Identification		0	D F	M F		Fragment offset	
TTL		Protocol		Checksum			
Source address							
Destination address							
Options (0~40)							
Payload (Data)							

단편화

- 수신 Host가 수용할 수 있는 크기를 넘어선 패킷을 작게 잘라 전송하는 것
- 잘게 나뉜 패킷들은 모두 동일한 ID 값을 가짐



02_IpFragments

```
/* Open the capture file */
if ((fp = pcap_open_offline(
    "C:\\SampleTraces\\ip-fragments.pcap",
    errbuf)) == NULL)
{
    fprintf(stderr, "\nUnable to open the file %s.\n",
        "C:\\SampleTraces\\ip-fragments.pcap");
    return -1;
}
```

02_IpFragments

```
if (pIpHeader->fragOffset & htons((short)0x2000) ||  
    ntohs(pIpHeader->fragOffset & (unsigned short)0xFF1F) > 0)  
{  
    printf("ID: %04X, Flags: %04X, Offset: %d, Protocol: 0x%02X\n",  
        ntohs(pIpHeader->id),  
        ntohs(pIpHeader->fragOffset & htons((short)0xE000)),  
        ntohs(pIpHeader->fragOffset & (unsigned short)0xFF1F) * 8,  
        pIpHeader->protocol);  
  
    printf("%d.%d.%d.%d -> %d.%d.%d.%d\n",  
        pIpHeader->srcIp[0], pIpHeader->srcIp[1],  
        pIpHeader->srcIp[2], pIpHeader->srcIp[3],
```

02_IpFragments

Microsoft Visual Studio 디버거

ID: 2D00, Flags: 2000, Offset: 0, Protocol: 0x01
10.1.0.1 -> 10.1.0.99

ID: 2D00, Flags: 2000, Offset: 1480, Protocol: 0x01
10.1.0.1 -> 10.1.0.99

ID: 2D00, Flags: 2000, Offset: 2960, Protocol: 0x01
10.1.0.1 -> 10.1.0.99

ID: 2D00, Flags: 0000, Offset: 0, Protocol: 0x01
10.1.0.1 -> 10.1.0.99

ID: 3100, Flags: 2000, Offset: 0, Protocol: 0x01
10.1.0.99 -> 10.1.0.1

3	1.064000	10.1.0.1	10.1.0.99	IPv4	1518
4	1.065000	10.1.0.1	10.1.0.99	IPv4	1518
5	1.066000	10.1.0.1	10.1.0.99	IPv4	1518
6	1.067000	10.1.0.1	10.1.0.99	ICMP	1206

Frame 4: 1518 bytes on wire (12144 bits), 1518 bytes captured (12144 bits)

Ethernet II, Src: NetworkGener_10:22:1b (00:00:65:10:22:1b), Dst: Runtop_e1:5a:80 (00:20:78:e1:5a:80)

Internet Protocol Version 4, Src: 10.1.0.1, Dst: 10.1.0.99

0100 = Version: 4

.... 0101 = Header Length: 20 bytes (5)

> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)

Total Length: 1500

Identification: 0x2d00 (11520)

> 001. = Flags: 0x1, More fragments

...0 0000 1011 1001 = Fragment Offset: 1480

Time to Live: 32

Protocol: ICMP (1)

TCP 헤더 분석

0 15 16 31

Source port					Destination port					
Sequence number										
Acknowledgement number										
Data	Rev.	C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window size
Checksum					Urgent pointer					
Options + padding										
Segment (Data)										

TCP 헤더 구조체 정의

```
typedef struct TcpHeader {  
    unsigned short srcPort;  
    unsigned short dstPort;  
    unsigned int seq;  
    unsigned int ack;  
    unsigned char data;  
    unsigned char flags;  
    unsigned short windowSize;  
    unsigned short checksum;  
    unsigned short urgent;  
} TcpHeader;  
#pragma pack(pop)
```


출발지/목적지 Port 번호

0										15										16										31															
Source port																				Destination port																									
Sequence number																																													
Acknowledgement number																																													
Data					Rev.					C W R		E C E		U R G		A C K		P S H		R S T		S Y N		F I N		Window size																			
Checksum																				Urgent pointer																									
Options + padding																																													
Segment (Data)																																													

0										15										16										31															
Source port																				Destination port																									
Sequence number																																													
Acknowledgement number																																													
Data					Rev.					C W R		E C E		U R G		A C K		P S H		R S T		S Y N		F I N		Window size																			
Checksum																				Urgent pointer																									
Options + padding																																													
Segment (Data)																																													

03_TcpSample

Microsoft Visual Studio 디버그

+

▼

—

□

×

```
192.168.1.182:1088 -> 161.58.73.170:80  
SYN  
161.58.73.170:80 -> 192.168.1.182:1088  
SYN + ACK  
192.168.1.182:1088 -> 161.58.73.170:80  
ACK  
192.168.1.182:1088 -> 161.58.73.170:80  
161.58.73.170:80 -> 192.168.1.182:1088  
ACK  
161.58.73.170:80 -> 192.168.1.182:1088  
192.168.1.182:1088 -> 161.58.73.170:80  
161.58.73.170:80 -> 192.168.1.182:1088  
192.168.1.182:1088 -> 161.58.73.170:80
```

03_TcpSample

```
EtherHeader* pEther = (EtherHeader*)pkt_data;
IpHeader* pIpHeader = (IpHeader*)(pkt_data + sizeof(EtherHeader));

if (pEther->type != 0x0008)
    return;

if (pIpHeader->protocol != 6)
    return;

int ipLen = (pIpHeader->verIhl & 0x0F) * 4;
TcpHeader* pTcp =
    (TcpHeader*)(pkt_data + sizeof(EtherHeader) + ipLen);
```

03_TcpSample

```
printf("%d.%d.%d.%d:%d -> %d.%d.%d.%d:%d\n",  
    pIpHeader->srcIp[0], pIpHeader->srcIp[1],  
    pIpHeader->srcIp[2], pIpHeader->srcIp[3],  
    ntohs(pTcp->srcPort),  
    pIpHeader->dstIp[0], pIpHeader->dstIp[1],  
    pIpHeader->dstIp[2], pIpHeader->dstIp[3],  
    ntohs(pTcp->dstPort)  
);
```

플래그 추출 및 TCP 상태

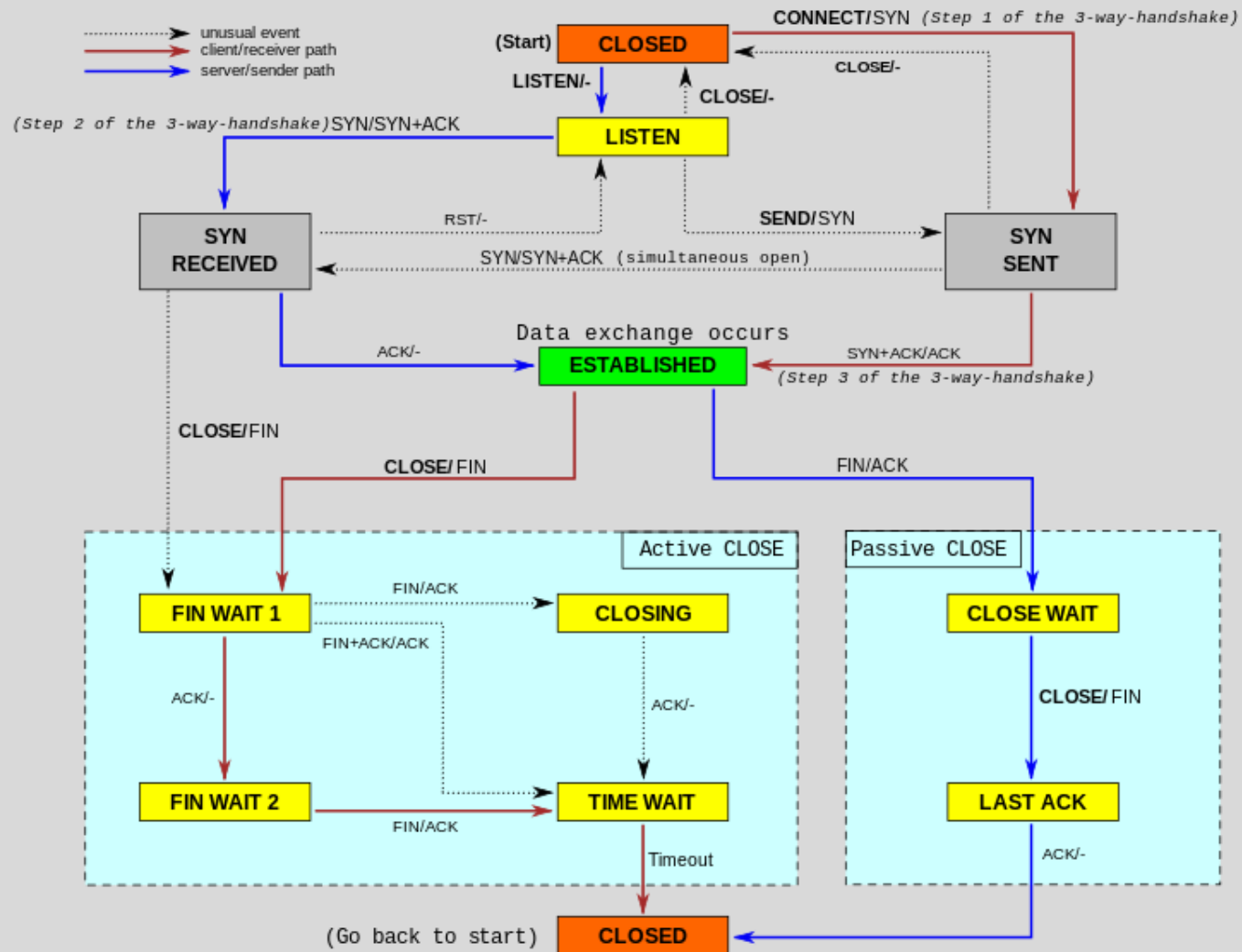
0										15										16										31									
Source port																				Destination port																			
Sequence number																																							
Acknowledgement number																																							
Data					Rev.					C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window size																					
Checksum																				Urgent pointer																			
Options + padding																																							
Segment (Data)																																							

Segment (Data)

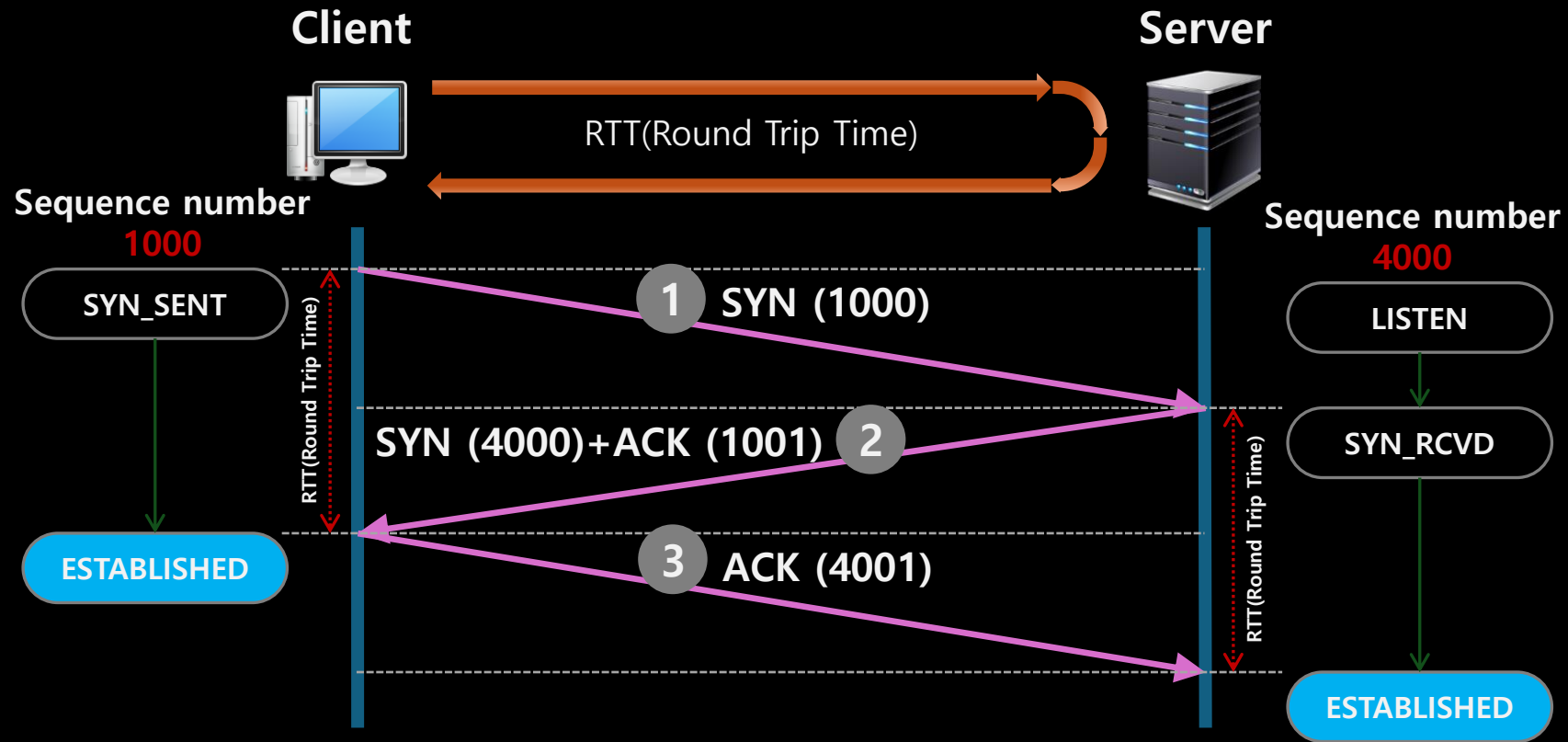
TCP 플래그

- CWR (Congestion Window Reduced)
- ECE (Explicit Congestion Notification–Echo)
- URG (Urgent)
- ACK (Acknowledgement)
- PSH (Push)
- RST (Reset)
- SYN (Synchronize)
- FIN (Finish)

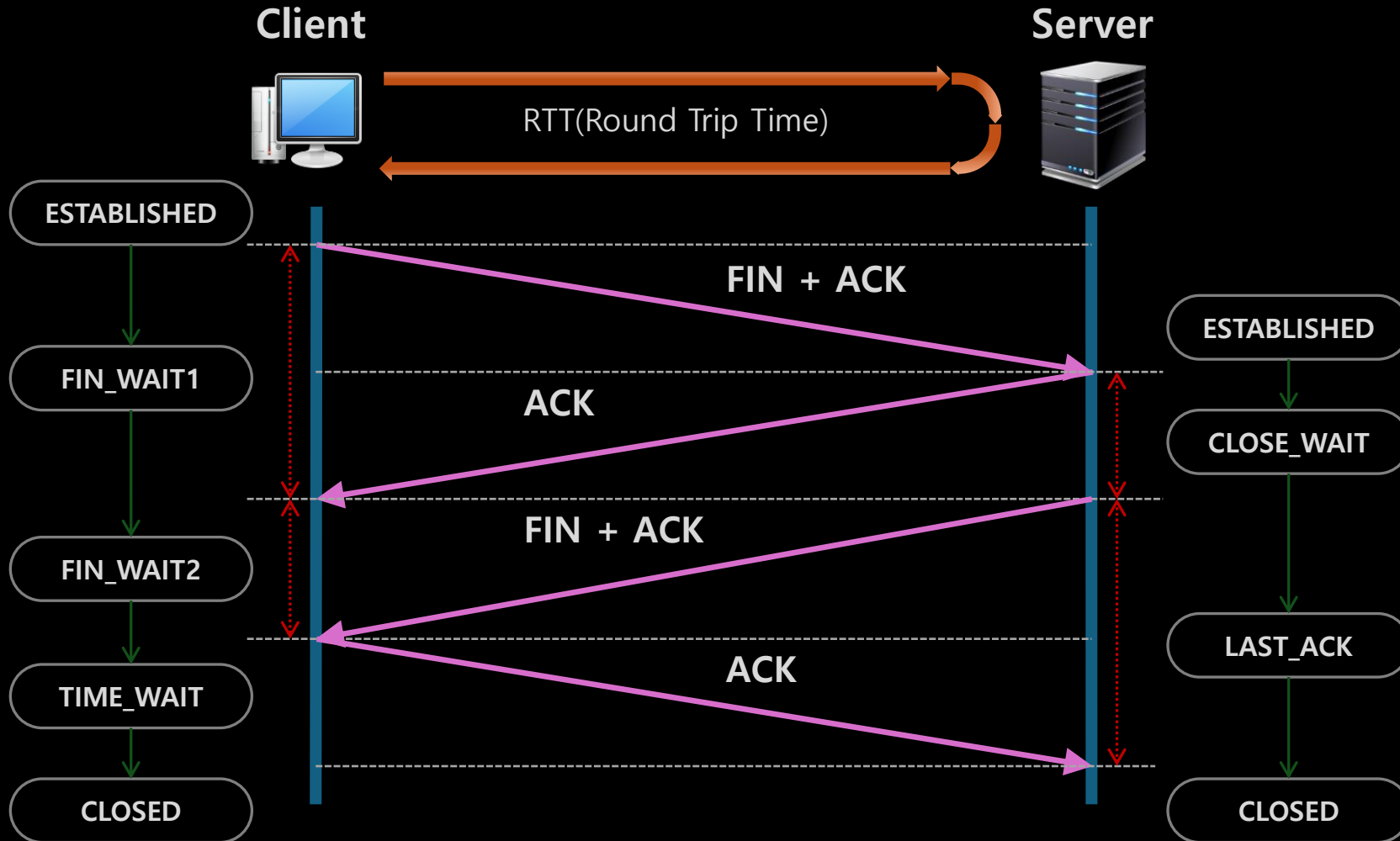
TCP (연결) 상태 변화



TCP 연결 과정 (3-way handshake)



TCP 연결 종료 과정 (4-way handshake)



03_TcpSample

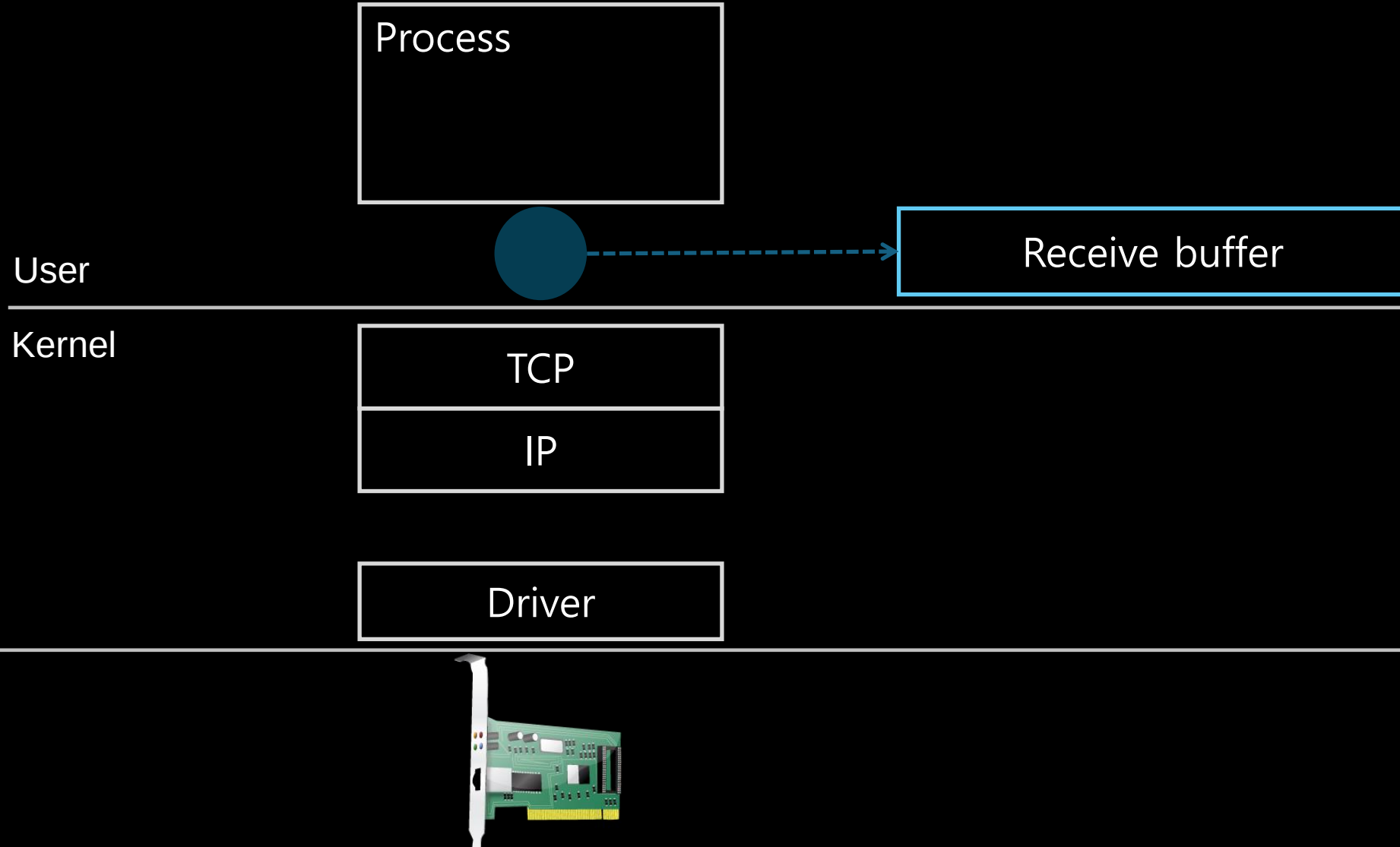
```
if (pTcp->flags == 0x02)
    puts("SYN");
else if (pTcp->flags == 0x12)
    puts("SYN + ACK");
else if (pTcp->flags == 0x10)
    puts("ACK");

if(pTcp->flags & 0x04)
    puts("*RST");
```

0										15 16										31									
Source port															Destination port														
Sequence number																													
Acknowledgement number																													
Data					Rev.					C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window size											
Checksum															Urgent pointer														
Options + padding																													
Segment (Data)																													

0															15 16															31														
Source port															Destination port																													
Sequence number																																												
Acknowledgement number																																												
Data					Rev.					C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window size																										
Checksum															Urgent pointer																													
Options + padding																																												
Segment (Data)																																												

Window size

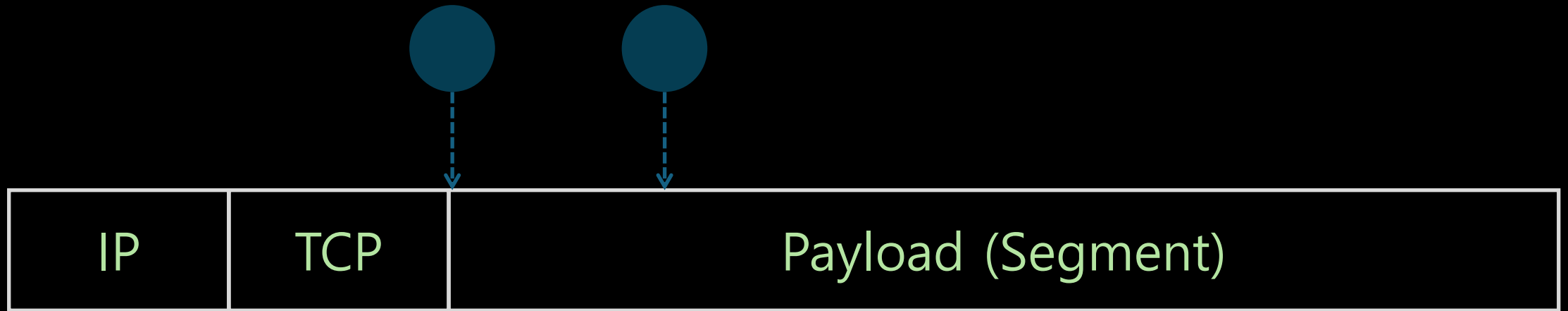


데이터 옵셋 및 페이로드 추출

0										15										16										31															
Source port																				Destination port																									
Sequence number																																													
Acknowledgement number																																													
Data					Rev.					C W R		E C E		U R G		A C K		P S H		R S T		S Y N		F I N		Window size																			
Checksum																				Urgent pointer																									
Options + padding																																													
Segment (Data)																																													

0										15										16										31															
Source port																				Destination port																									
Sequence number																																													
Acknowledgement number																																													
Data					Rev.					C W R		E C E		U R G		A C K		P S H		R S T		S Y N		F I N		Window size																			
Checksum																				Urgent pointer																									
Options + padding																																													
Segment (Data)																																													

데이터 옵셋 및 페이로드 추출





```
C:\Users\User\Desktop\03_Tc1 x + v - □ x  
*** The server has started. ***  
***New client***  
***New client***  
192.168.40.1: Hello  
192.168.40.1: Hi
```

```
F:\WNpcap으로 패킷 분석기 작 x + v - □ x  
채팅을 시작합니다. 메시지를 입력하세요.  
Hello  
-> Hello  
-> Hi
```

```
F:\WNpcap으로 패킷 분석기 작 x + v - □ x  
채팅을 시작합니다. 메시지를 입력하세요.  
-> Hello  
Hi  
-> Hi
```

03_TcpChatMonitor

```
EtherHeader* pEther = (EtherHeader*)pkt_data;
IpHeader* pIpHeader = (IpHeader*)(pkt_data + sizeof(EtherHeader));

if (pEther->type != 0x0008)
    return;

if (pIpHeader->protocol != 6)
    return;

int ipHeaderLen = (pIpHeader->verIhl & 0x0F) * 4;
TcpHeader* pTcp =
    (TcpHeader*)(pkt_data + sizeof(EtherHeader) + ipHeaderLen);
```


03_TcpChatMonitor

```
if (ntohs(pTcp->srcPort) != 25000 &&
    ntohs(pTcp->dstPort) != 25000)
    return;

printf("%d.%d.%d.%d:%d -> %d.%d.%d.%d:%d\n",
    pIpHeader->srcIp[0], pIpHeader->srcIp[1],
    pIpHeader->srcIp[2], pIpHeader->srcIp[3],
    ntohs(pTcp->srcPort),
    pIpHeader->dstIp[0], pIpHeader->dstIp[1],
    pIpHeader->dstIp[2], pIpHeader->dstIp[3],
    ntohs(pTcp->dstPort)
);
```

03_TcpChatMonitor

```
int tcpHeaderSize = ((pTcp->data >> 4 & 0x0F) * 4);
char* pPayload = (char*)(pkt_data + sizeof(EtherHeader) +
    ipHeaderLen + tcpHeaderSize);

printf("Segment size: %d(Frame length: %d)\n",
    ntohs(pIpHeader->length) - ipHeaderLen - tcpHeaderSize,
    header->len);

char szMessage[2048] = { 0 };
memcpy_s(szMessage, sizeof(szMessage), pPayload,
    .....ntohs(pIpHeader->length) - ipHeaderLen - tcpHeaderSize);
puts(szMessage);
```

03_TcpChatMonitor

```
F:\Npcap으로 패킷 분석기 작 x + v
192.168.40.128:25000 -> 192.168.40.1:58520
Segment size: 3(Frame length: 57)
Hi
192.168.40.1:58520 -> 192.168.40.128:25000
Segment size: 0(Frame length: 54)

192.168.40.1:58520 -> 192.168.40.128:25000
Segment size: 12(Frame length: 66)
Hello npcap
192.168.40.128:25000 -> 192.168.40.1:58520
Segment size: 12(Frame length: 66)
Hello npcap
192.168.40.1:58520 -> 192.168.40.128:25000
Segment size: 0(Frame length: 54)

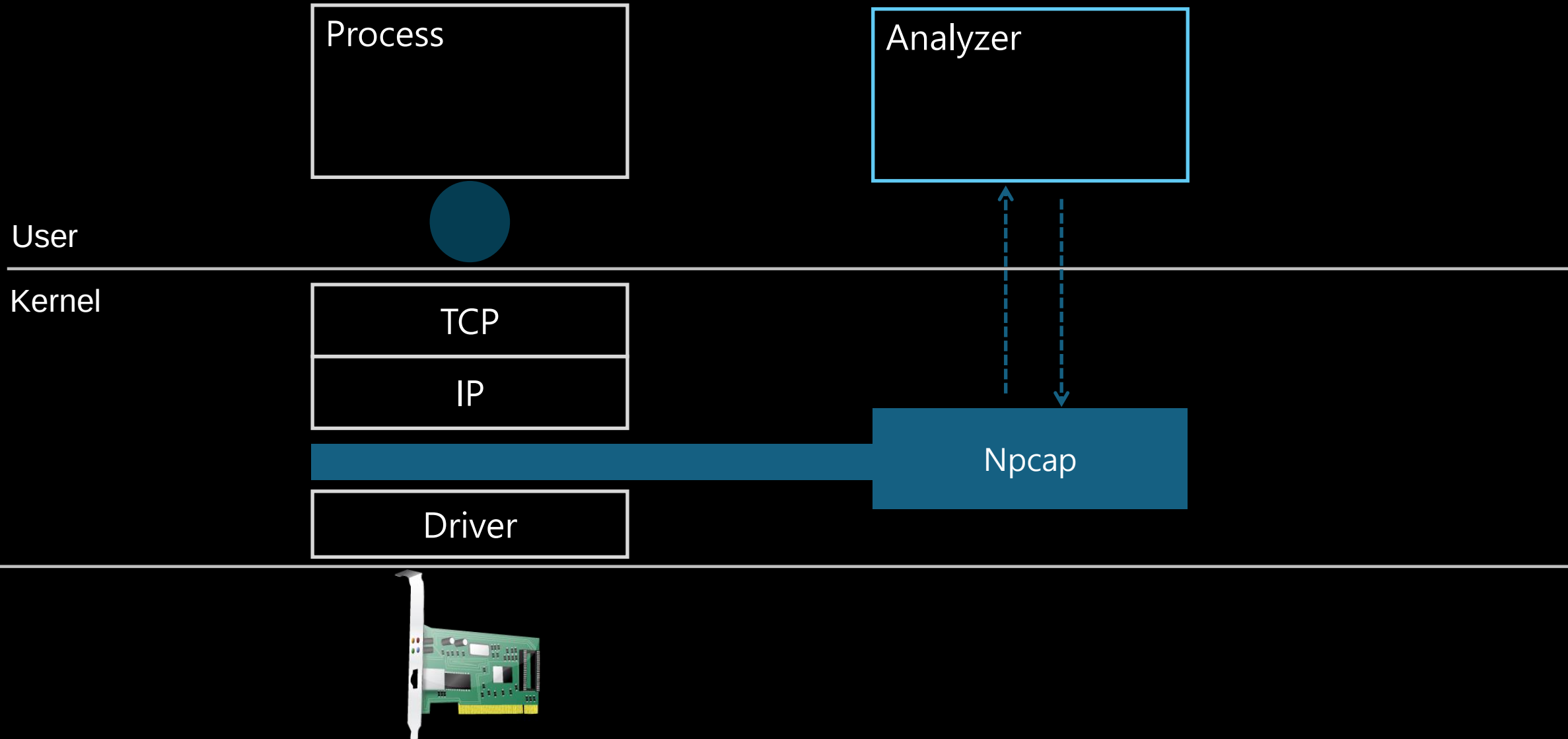
|
```

패킷 생성 및 전송

Ethernet 채팅 프로그램 개발

- IP 패킷의 경우 L2 ethernet type 값은 0x0800
- 임의의 type 값을 주어 L2 Frame을 식별
- L3 데이터에 별도 프로토콜 없이 송/수신 메시지를 직접 저장
- 출발지/목적지 MAC 주소는 자신의 시스템 상황에 맞춰 적절히 설정
 - 예제에서는 브로드캐스트 처리

메시지 직접 전달





Capturing from Ethernet0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	00:00:00	00:00:00	Broadcast	60	Ethernet II
2	9.889186	00:00:00	00:00:00	Broadcast	60	Ethernet II
3	12.163008	192.168	192.168	IP	60	Ethernet II
4	13.163230	192.168	192.168	IP	60	Ethernet II
5	14.163283	192.168	192.168	IP	60	Ethernet II
6	15.164635	192.168	192.168	IP	60	Ethernet II

> Frame 2: 60 bytes on wire (48 bytes captured) on interface 0
> Ethernet II, Src: 00:00:00:00:00:00, Dst: 00:00:00:00:00:00
> Data (46 bytes)

0000	ff	ff	ff	ff	ff	ff	00
0010	6c	6c	6f	20	65	74	68
0020	00	00	00	00	00	00	00
0030	00	00	00	00	00	00	00

Ethernet0: <live capture in progress>

C:\Users\User\Desktop\04_Etl

1. \Device\NPF_{21490330-D19C-441B-8B6B-A27AFE9A559D} (WAN Miniport (Network Monitor))
2. \Device\NPF_{48ED3877-089D-411F-B8A1-E70A78F24176} (WAN Miniport (IPv6))
3. \Device\NPF_{7D1EAAA5-8580-4626-B6C3-D7A84DBD8A85} (WAN Miniport (IP))
4. \Device\NPF_{D49874D5-043D-4A44-A4DA-162123D60006} (Hyper-V Virtual Ethernet Adapter)
5. \Device\NPF_{8497F061-786F-4969-81CB-5A8537264D12} (Intel(R) PRO/1000 MT Network Connection #2)
6. \Device\NPF_{7DECE69D-9E56-4624-9EE0-A1D20A7CF5E3} (Intel(R) PRO/1000 MT Network Connection)
7. \Device\NPF_Loopback (Adapter for loopback traffic capture)

Enter the interface number (1-7):6
[Ethernet message receiver]

listening on Intel(R) PRO/1000 MT Network Connection...
Ethernet message: test
Ethernet message: Hello ethernet





Capturing from Ethernet0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	00:00:00_00:00:...	Broadcast	0x00...	60	Ethernet II
2	9.889186	00:00:00_00:00:...	Broadcast	0x00...	60	Ethernet II
3	12.163008	192.168.40.1	239.255.255.250	SSDP	217	M-SEARCH * HTTP/1.
4	13.163230	192.168.40.1	239.255.255.250	SSDP	217	M-SEARCH * HTTP/1.
5	14.163283	192.168.40.1	239.255.255.250	SSDP	217	M-SEARCH * HTTP/1.
6	15.164635	192.168.40.1	239.255.255.250	SSDP	217	M-SEARCH * HTTP/1.

> Frame 2: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface \D

> Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: Broadcast (ff:ff:ff:ff:ff:ff)

> Data (46 bytes)

Offset	Hex	ASCII
0000	ff ff ff ff ff ff 00 00 00 00 00 00 00 48 65	He
0010	6c 6c 6f 20 65 74 68 65 72 65 6e 74 00 00 00 00	llo ethe rent...
0020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Ethernet0: <live capture in progress>

Packets: 6 · Displayed: 6 (100.0%)

Profile: Default

```
Monitor))
net Adapter)
Network Connection #2)
Network Connection)
```


04_EtherSender

```
unsigned char frameData[1514] = { 0 };  
int msgSize = 0;  
EtherHeader* pEtherHeader = (EtherHeader*)frameData;  
pEtherHeader->dstMac[0] = 0xFF; pEtherHeader->dstMac[1] = 0xFF;  
pEtherHeader->dstMac[2] = 0xFF; pEtherHeader->dstMac[3] = 0xFF;  
pEtherHeader->dstMac[4] = 0xFF; pEtherHeader->dstMac[5] = 0xFF;  
pEtherHeader->type = 0x0000;  
char *pData = (char*)frameData + sizeof(EtherHeader);  
char szInput[1024] = { 0 };
```

04_EtherSender

```
while (1)
{
    memset(pData, 0, 1514 - sizeof(EtherHeader));
    memset(szInput, 0, sizeof(szInput));
    printf("Message: ");
    gets_s(szInput, sizeof(szInput));

    if (strcmp(szInput, "exit") == 0)
        break;
```

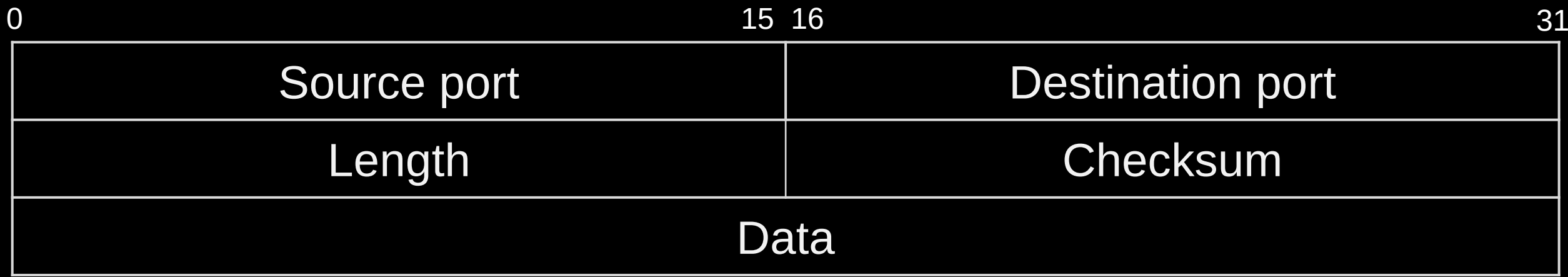
04_EtherSender

```
msgSize = strlen(szInput);  
strcpy_s(pData, msgSize + 1, szInput);  
  
/* Send down the packet */  
if (pcap_sendpacket(adhandle, // Adapter  
    frameData, // buffer with the packet  
    sizeof(EtherHeader) + msgSize // size  
    ) != 0)
```

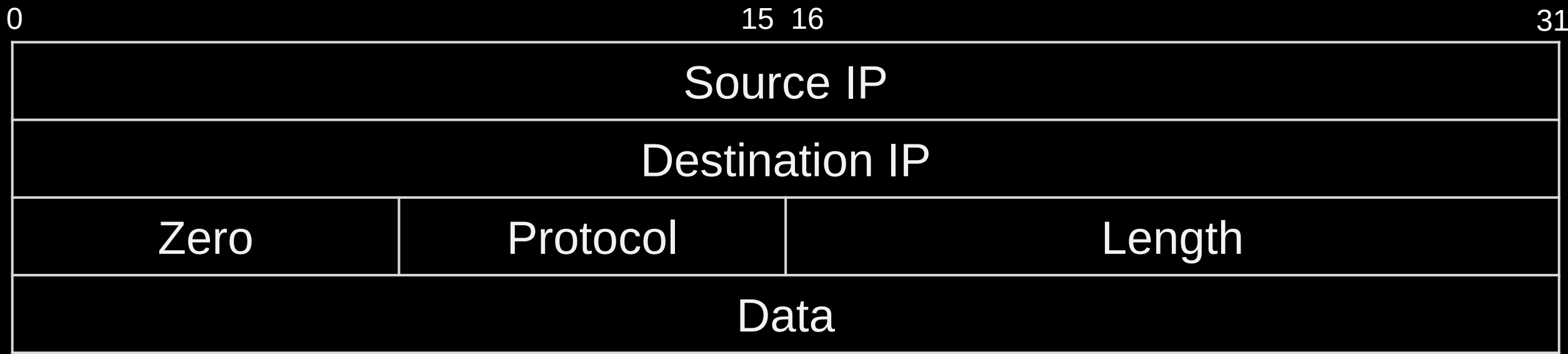
04_EtherReceiver

```
EtherHeader* pEther = (EtherHeader*)pkt_data;  
if (pEther->type != 0x0000)  
    return;  
  
char szBuffer[2048] = { 0 };  
memcpy_s(szBuffer, sizeof(szBuffer),  
         pkt_data + sizeof(EtherHeader),  
         header->len - sizeof(EtherHeader));  
  
printf("Ethernet message: %s\n", szBuffer);
```

UDP 헤더 형식



UDP 가상 헤더 형식



04_UdpIpChecksum

```
typedef struct UdpHeader {  
    unsigned short srcPort;  
    unsigned short dstPort;  
    unsigned short length;  
    unsigned short checksum;  
} UdpHeader;
```

```
typedef struct PseudoHeader {  
    unsigned int srcIp;  
    unsigned int dstIp;  
    unsigned char zero;  
    unsigned char protocol;  
    unsigned short length;  
} PseudoHeader;  
#pragma pack(pop)
```

04_UdpIpChecksum

```
unsigned short CalcChecksumIp(IpHeader* pIpHeader)
{
    unsigned char ihl = (pIpHeader->verIhl & 0x0F) << 2;
    unsigned short wData[30] = { 0 };
    unsigned int dwSum = 0;

    memcpy(wData, (BYTE*)pIpHeader, ihl);
    //((IpHeader*)wData)->checksum = 0x0000;
```


04_UdplpChecksum

```
for (int i = 0; i < ihl / 2; i++)
{
    if (i != 5)
        dwSum += wData[i];

    if (dwSum & 0xFFFF0000)
    {
        dwSum &= 0x0000FFFF;
        dwSum++;
    }
}

return ~(dwSum & 0x0000FFFF);
```

04_UdpIpChecksum

```
unsigned short CalcChecksumUdp( IpHeader* pIpHeader,
                                UdpHeader* pUdpHeader)
{
    PseudoHeader    pseudoHeader = { 0 };
    unsigned short* pwPseudoHeader = (unsigned short*)&pseudoHeader;
    unsigned short* pwDatagram = (unsigned short*)pUdpHeader;
    int              nPseudoHeaderSize = 6; //WORD 6개 배열
    int              nDatagramSize = 0; //헤더 포함 데이터그램 크기

    UINT32           dwSum = 0;
    int              nLengthOfArray = 0;
```

04_UdpIpChecksum

```
pseudoHeader.srcIp = *(unsigned int*)pIpHeader->srcIp;
pseudoHeader.dstIp = *(unsigned int*)pIpHeader->dstIp;
pseudoHeader.zero = 0;
pseudoHeader.protocol = 17;
pseudoHeader.length = pUdpHeader->length;

nDatagramSize = ntohs(pseudoHeader.length);

if (nDatagramSize % 2)
    nLengthOfArray = nDatagramSize / 2 + 1;
else
    nLengthOfArray = nDatagramSize / 2;
```

04_UdplpChecksum

```
for (int i = 0; i < nPseudoHeaderSize; i++)
{
    dwSum += pwPseudoHeader[i];
    if (dwSum & 0xFFFF0000)
    {
        dwSum &= 0x0000FFFF;
        dwSum++;
    }
}
```

04_UdplpChecksum

```
//((UdpHeader*)wData)->checksum = 0x0000;
for (int i = 0; i < nLengthOfArray; i++)
{
    if (i != 3)
        dwSum += pwDatagram[i];
    if (dwSum & 0xFFFF0000)
    {
        dwSum &= 0x0000FFFF;
        dwSum++;
    }
}

return (USHORT)~(dwSum & 0x0000FFFF);
```

Capturing from Ethernet0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

udp.port == 26001

No.	Time	Source	Destination	Protocol	Length	Info
5	8.026054	1.1.1.1	192.168.40.128	UDP	60	26780 → 26001 Len=
6	15.418313	1.1.1.1	192.168.40.128	UDP	60	26780 → 26001 Len=

Checksum: 0x67cc [unverified]
[Checksum Status: Unverified]
[Stream index: 1]
> [Timestamps]
UDP payload (10 bytes)
Data (10 bytes)
Data: 554450207061636b6574

0000	00	0c	29	35	0d	eb	00	50
0010	00	26	12	34	40	00	ff	11
0020	28	80	68	9c	65	91	00	12
0030	63	6b	65	74	00	00	00	00

Data (data.data), 10 bytes

C:\Users\User\Desktop\04_Ud

```
1.1.1.1: UDP packet
1.1.1.1: Hello
```

F:\Npcap으로 패킷 분석기 작

```
1. \Device\NPF_{8443A134-778D-4CB7-BE5D-B5ED95DCBCEF} (WAN Miniport (Network Monitor))
2. \Device\NPF_{EB295B87-F0E2-4666-8E90-0B07A5E39CE9} (WAN Miniport (IPv6))
3. \Device\NPF_{B50665C3-9648-40BA-9AEE-DBF26D528A5D} (WAN Miniport (IP))
4. \Device\NPF_{75CD8E73-599F-49D0-8ABB-176CD8AF12D1} (VMware Virtual Ethernet Adapter for VMn
et8)
5. \Device\NPF_{B795FF90-8B10-4783-8A5F-2AF9EAA98BA6} (VMware Virtual Ethernet Adapter for VMn
et1)
6. \Device\NPF_{3BA0EEAB-694A-4805-A7CE-FCDA37220C54} (Intel(R) Ethernet Connection (7) I219-V
)
7. \Device\NPF_Loopback (Adapter for loopback traffic capture)
Enter the interface number (1-7):5
[Ethernet message sender]
Message: UDP packet
Message: Hello
Message: |
```

UDP 데이터그램 생성 및 전송

```
unsigned char frameData[1514] = { 0 };
int msgSize = 0;
EtherHeader* pEtherHeader = (EtherHeader*)frameData;
pEtherHeader->dstMac[0] = 0x00; pEtherHeader->dstMac[1] = 0x0C;
pEtherHeader->dstMac[2] = 0x29; pEtherHeader->dstMac[3] = 0x35;
pEtherHeader->dstMac[4] = 0x0D; pEtherHeader->dstMac[5] = 0xEB;

pEtherHeader->srcMac[0] = 0x00; pEtherHeader->srcMac[1] = 0x50;
pEtherHeader->srcMac[2] = 0x56; pEtherHeader->srcMac[3] = 0xC0;
pEtherHeader->srcMac[4] = 0x00; pEtherHeader->srcMac[5] = 0x08;

pEtherHeader->type = 0x0008;
```


UDP 데이터그램 생성 및 전송

```
IpHeader* pIpHeader = (IpHeader*)(frameData + sizeof(EtherHeader));  
pIpHeader->verIhl = 0x45;  
pIpHeader->tos = 0x00;  
pIpHeader->length = 0;  
pIpHeader->id = 0x3412;  
pIpHeader->fragOffset = 0x0040; //DF  
pIpHeader->ttl = 0xFF;  
pIpHeader->protocol = 17; // UDP  
pIpHeader->checksum = 0x0000;
```

UDP 데이터그램 생성 및 전송

```
pIpHeader->srcIp[0] = 1;  
pIpHeader->srcIp[1] = 1;  
pIpHeader->srcIp[2] = 1;  
pIpHeader->srcIp[3] = 1;
```

```
pIpHeader->dstIp[0] = 192;  
pIpHeader->dstIp[1] = 168;  
pIpHeader->dstIp[2] = 40;  
pIpHeader->dstIp[3] = 128;
```

UDP 데이터그램 생성 및 전송

```
int ipHeaderLen = 20;
UdpHeader* pUdpHeader =
    (UdpHeader*)(frameData + sizeof(EtherHeader) + ipHeaderLen);

pUdpHeader->srcPort = htons(26780);
pUdpHeader->dstPort = htons(26001);
pUdpHeader->length = 0;
pUdpHeader->checksum = 0x0000;
```

UDP 데이터그램 생성 및 전송

```
char szInput[1024];
char *pPayload = (char*)(frameData + sizeof(EtherHeader) +
    ipHeaderLen + sizeof(UdpHeader));
while (1)
{
    memset(szInput, 0, sizeof(szInput));
    printf("Message: ");
    gets_s(szInput, sizeof(szInput));
    if (strcmp(szInput, "exit") == 0)
        break;
```

UDP 데이터그램 생성 및 전송

```
msgSize = (int)strlen(szInput);
strcpy_s(pPayload, msgSize + 1, szInput);

pUdpHeader->length = htons(
    (unsigned short)sizeof(UdpHeader) + msgSize);
pIpHeader->length = htons(
    (unsigned short)(sizeof(IpHeader) +
        sizeof(UdpHeader) + msgSize));

pIpHeader->checksum = CalcChecksumIp(pIpHeader);
pUdpHeader->checksum = CalcChecksumUdp(pIpHeader, pUdpHeader);
```

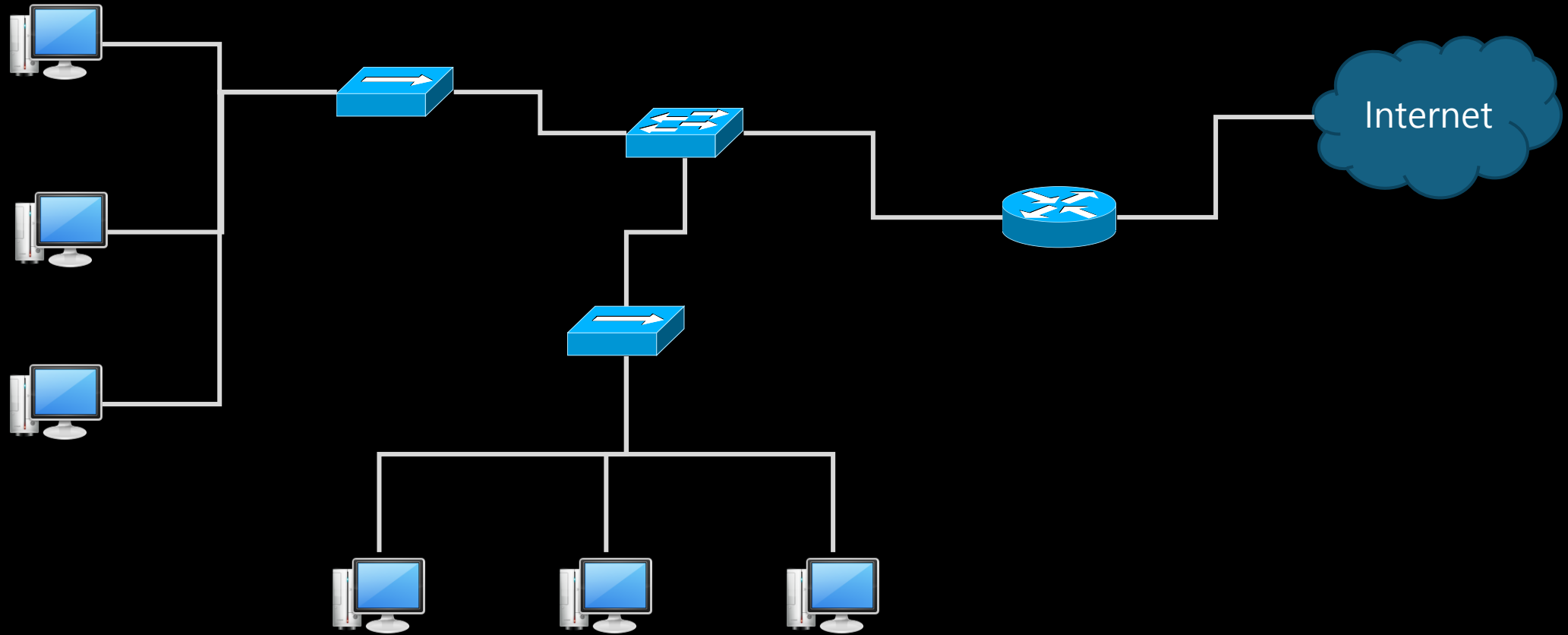
UDP 데이터그램 생성 및 전송

```
/* Send down the packet */  
if (pcap_sendpacket(adhandle, // Adapter  
    frameData, // buffer with the packet  
    sizeof(EtherHeader) + sizeof(IpHeader) +  
    sizeof(UdpHeader) + msgSize // size  
) != 0)
```

TCP 연결 강제 종료

- 서로 TCP 연결된 Client와 Server의 연결을 제3의 Host에서 종료 시키는 방법
- TCP 세션 정보가 일치하는 위조 패킷을 생성
 - 세션 정보는 출발지/목적지 IP주소 및 Port번호
 - RST 플래그 설정
 - 반드시 Sequence 번호가 일치해야 작동
- 연결이 끊인 Client는 상황을 인지하지 못하는 현상 관찰
 - 연결에 대한 본질적 의미 재정의

TCP '연결' 이라는 착각



04_TcpReset

```
unsigned char frameData[1514] = { 0 };
int msgSize = 0;
EtherHeader* pEtherHeader = (EtherHeader*)frameData;
pEtherHeader->dstMac[0] = 0x00; pEtherHeader->dstMac[1] = 0x0C;
pEtherHeader->dstMac[2] = 0x29; pEtherHeader->dstMac[3] = 0x35;
pEtherHeader->dstMac[4] = 0x0D; pEtherHeader->dstMac[5] = 0xEB;

pEtherHeader->srcMac[0] = 0x00; pEtherHeader->srcMac[1] = 0x50;
pEtherHeader->srcMac[2] = 0x56; pEtherHeader->srcMac[3] = 0xC0;
pEtherHeader->srcMac[4] = 0x00; pEtherHeader->srcMac[5] = 0x08;

pEtherHeader->type = 0x0008;
```

04_TcpReset

```
IpHeader* pIpHeader = (IpHeader*)(frameData + sizeof(EtherHeader));  
pIpHeader->verIhl = 0x45;  
pIpHeader->tos = 0x00;  
pIpHeader->length = htons(40);  
pIpHeader->id = 0x3412;  
pIpHeader->fragOffset = 0x0040; //DF  
pIpHeader->ttl = 0xFF;  
pIpHeader->protocol = 6; // TCP  
pIpHeader->checksum = 0x0000;
```

04_TcpReset

```
pIpHeader->srcIp[0] = 192;  
pIpHeader->srcIp[1] = 168;  
pIpHeader->srcIp[2] = 40;  
pIpHeader->srcIp[3] = 1;
```

```
pIpHeader->dstIp[0] = 192;  
pIpHeader->dstIp[1] = 168;  
pIpHeader->dstIp[2] = 40;  
pIpHeader->dstIp[3] = 128;
```

04_TcpReset

```
int ipHeaderLen = 20;
TcpHeader* pTcpHeader =
    (TcpHeader*)(frameData + sizeof(EtherHeader) + ipHeaderLen);

pTcpHeader->srcPort = htons(55881); //반드시 일치
pTcpHeader->dstPort = htons(25000);
pTcpHeader->seq = htonl(0xad4b4ed8); //반드시 일치
pTcpHeader->ack = 0;
pTcpHeader->data = 0x50;
pTcpHeader->flags = 0x04; //RST
pTcpHeader->windowSize = 0;
pTcpHeader->checksum = 0x0000;
pTcpHeader->urgent = 0;
```

04_TcpReset

```
pIpHeader->checksum = CalcChecksumIp(pIpHeader);
pTcpHeader->checksum = CalcChecksumTcp(pIpHeader, pTcpHeader);

/* Send down the packet */
if (pcap_sendpacket(adhandle, // Adapter
    frameData, // buffer with the packet
    sizeof(EtherHeader) + sizeof(IpHeader) + sizeof(TcpHeader)
) != 0)
```

생각할 문제와 프로젝트 제안

HTTP 트래픽 추출

- TCP/IP를 기반 프로토콜로 사용하는 HTTP 트래픽 분석
 - TCP 세그먼트 데이터 내부의 HTTP 헤더 분석
 - SPI(Shallow Packet Inspection)
 - TCP 80번 포트 통신만 한정
- HTTP가 ASCII 코드로 구성되어 있음을 활용
 - 예제에서는 전체 HTTP 트래픽을 단순 출력
 - 필드 단위로 정보를 추출하는 기능 추가 필요
 - HTTP는 0x50545448

HTTP 트래픽 추출 (05_HttpSample)

```
/* Open the capture file */
if ((fp = pcap_open_offline(
    "C:\\SampleTraces\\http-browse-ok.pcap",
    errbuf)) == NULL)
{
    fprintf(stderr, "\nUnable to open the file %s.\n",
        "C:\\SampleTraces\\http-browse-ok.pcap");
    return -1;
}
```


HTTP 트래픽 추출

```
EtherHeader* pEther = (EtherHeader*)pkt_data;
IpHeader* pIpHeader = (IpHeader*)(pkt_data + sizeof(EtherHeader));

if (pEther->type != 0x0008)
    return;

if (pIpHeader->protocol != 6)
    return;

int ipLen = (pIpHeader->verIhl & 0x0F) * 4;
TcpHeader* pTcp =
    (TcpHeader*)(pkt_data + sizeof(EtherHeader) + ipLen);

int tcpLen = ((pTcp->data >> 4) * 4);
```

HTTP 트래픽 추출

```
if (ntohs(pTcp->dstPort) == 80 &&
    ntohs(pIpHeader->length) > 50)
{
    char* pHttp = ((char*)pTcp) + tcpLen;

    printf("%d.%d.%d.%d:%d -> %d.%d.%d.%d:%d\n%s\n",
        pIpHeader->srcIp[0], pIpHeader->srcIp[1],
        pIpHeader->srcIp[2], pIpHeader->srcIp[3],
        ntohs(pTcp->srcPort),
        pIpHeader->dstIp[0], pIpHeader->dstIp[1],
        pIpHeader->dstIp[2], pIpHeader->dstIp[3],
        ntohs(pTcp->dstPort),
        pHttp
    );
}
```

HTTP 트래픽 추출

Microsoft Visual Studio 디버그

```
192.168.1.182:1088 -> 161.58.73.170:80
GET / HTTP/1.1
Accept: */*
Accept-Language: en-us
Accept-Encoding: gzip, deflate
If-Modified-Since: Sat, 16 Mar 2002 07:16:37 GMT; length=69556
User-Agent: Mozilla/4.0 (compatible; MSIE 5.5; Windows NT 5.0)
Host: www.packet-level.com
Connection: Keep-Alive
```

```
?5?
192.168.1.182:1089 -> 161.58.73.170:80
GET /scripts/behActions.js HTTP/1.1
Accept: */*
Referer: http://www.packet-level.com/
Accept-Language: en-us
Accept-Encoding: gzip, deflate
If-Modified-Since: Sat, 03 Mar 2001 22:39:34 GMT; length=6000
User-Agent: Mozilla/4.0 (compatible; MSIE 5.5; Windows NT 5.0)
Host: www.packet-level.com
Connection: Keep-Alive
```

유해사이트 차단 시스템 제작

- 특정 PC가 임의의 사이트를 방문하는 것을 탐지하고 거짓 응답을 보내 접속을 통제하는 구조
- HTTP 통제
 - 거짓 HTTP 응답을 회신해 다른 사이트로 접속 Redirect
 - HTTPS에 대해서는 SNI(Server Name Indication)필드를 확인하는 방식
- DNS 통제
 - DNS 질의에 대해 거짓 응답

*이더넷 2

파일(F) 편집(E) 보기(V) 이동(G) 캡처(C) 분석(A) 통계(S) 전화(Y) 무선(W) 도구(T) 도움말(H)

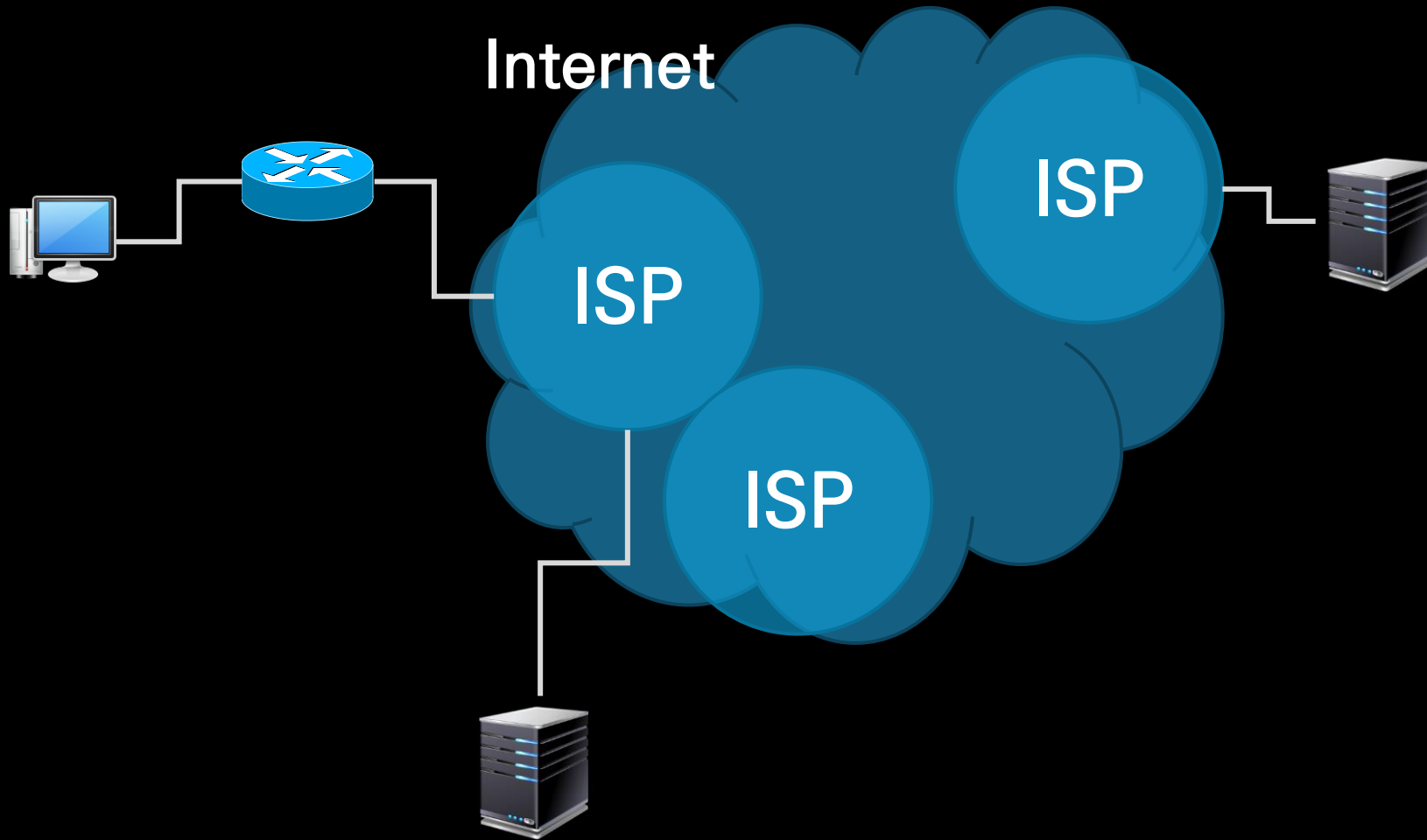
tls contains "naver"

No.	Time	Source	Destination	Protocol	Length	Host	Info
252	7.948730	192.168.50.42	104.109.240.195	TLSv1.2	541		Client H

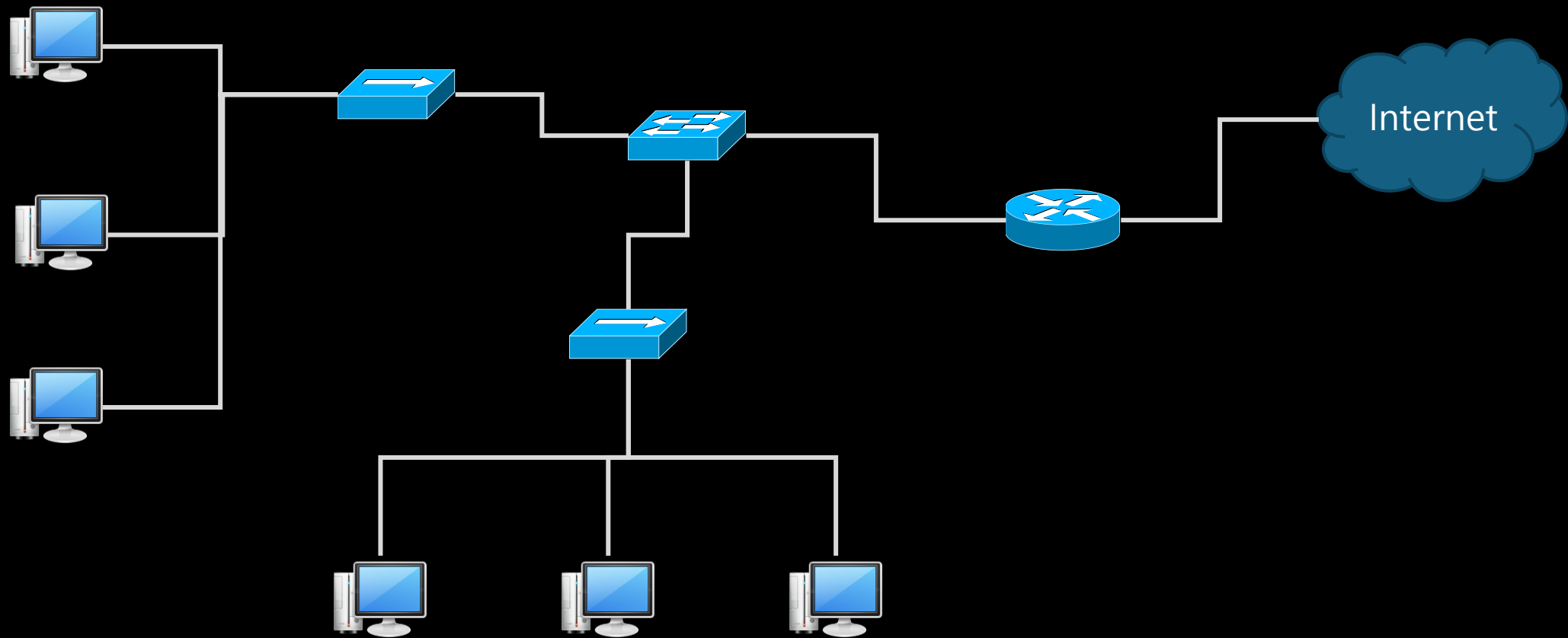
Transport Layer Security

- ▼ TLSv1.2 Record Layer: Handshake Protocol: Client Hello
 - Content Type: Handshake (22)
 - Version: TLS 1.0 (0x0301)
 - Length: 1942
- ▼ Handshake Protocol: Client Hello
 - Handshake Type: Client Hello (1)
 - Length: 1938
 - Version: TLS 1.2 (0x0303)
 - > Random: 84db52a72e93b76d9c7a71906401a37dff3f4f1e456fac4e5e258855167ef7aa
 - Session ID Length: 32
 - Session ID: 4826d3d1026271731e28cf1f1e1bc71f1ea7c4d0f7af38ce8cdc89cd076495ae
 - Cipher Suites Length: 32
 - > Cipher Suites (16 suites)
 - Compression Methods Length: 1
 - > Compression Methods (1 method)
 - Extensions Length: 1833
 - > Extension: Reserved (GREASE) (len=0)
 - > Extension: server_name (len=18) name=www.naver.com

유해사이트 차단 시스템 제작



유해사이트 차단 시스템 제작



ARP 제어 방식 통제

- ARP 테이블을 동적 업데이트 하는 기본 설정을 활용
 - 정적 설정인 경우 작동하지 않음
- 거짓 ARP 응답을 보내 인터넷을 차단시키려는 특정 Host의 Gateway MAC주소를 변경
 - 사실 상 ARP spoofing 공격과 동일
 - 비표준 방식 NAC
 - L2 스위치가 오작동하거나 다운될 수 있음에 주의

ARP 테이블 확인

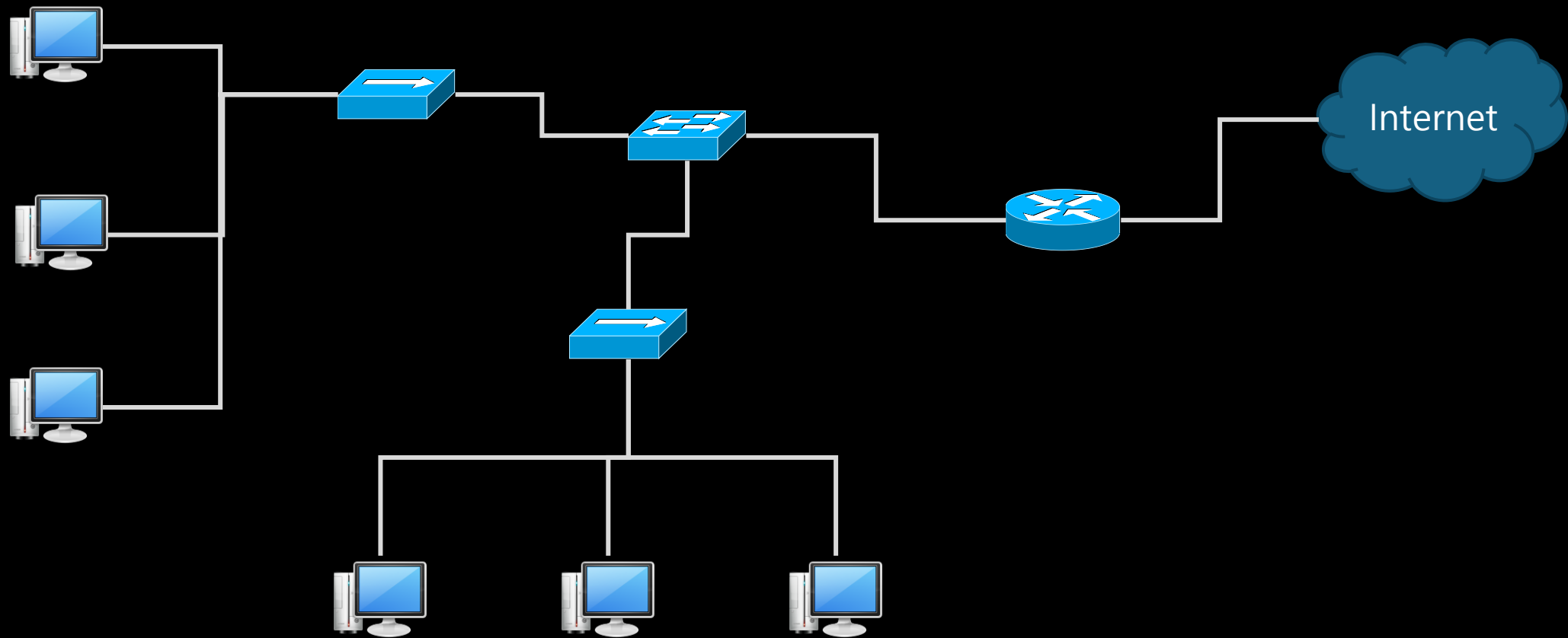
```
명령 프롬프트
Microsoft Windows [Version 10.0.22631.3447]
(c) Microsoft Corporation. All rights reserved.

C:\Users\cx853>arp -a

인터페이스 : 192.168.50.42 --- 0x5
  인터넷 주소      물리적 주소      유형
192.168.50.1      0c-9d-92-02-b9-80   동적
192.168.50.156    84-25-19-65-3e-25   동적
192.168.50.255    ff-ff-ff-ff-ff-ff   정적
224.0.0.22        01-00-5e-00-00-16   정적
224.0.0.251       01-00-5e-00-00-fb   정적
224.0.0.252       01-00-5e-00-00-fc   정적
239.255.255.250   01-00-5e-7f-ff-fa   정적
255.255.255.255   ff-ff-ff-ff-ff-ff   정적

인터페이스 : 192.168.253.1 --- 0x8
```

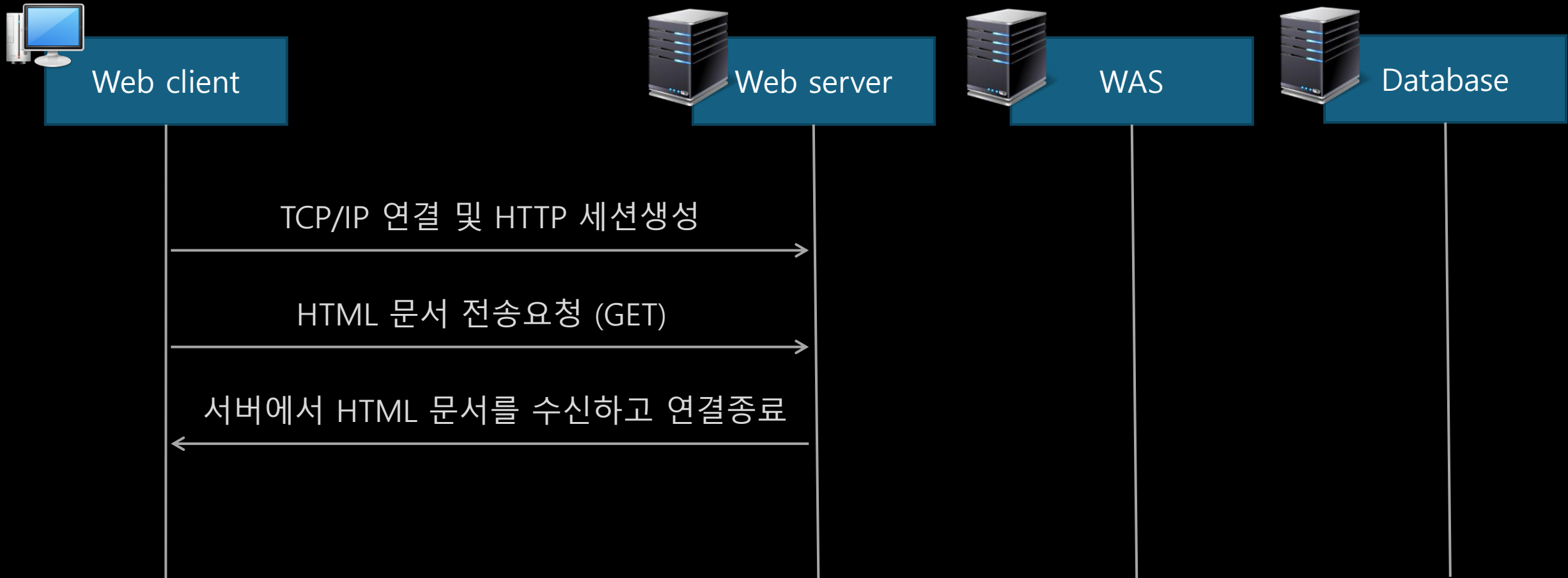
PC 인터넷 접근 제어



네트워크 기반 APM 제작

- HTTP Client의 요청(GET)을 탐지하고 그에 대한 응답 시간을 측정
- HTTPS가 아닌 HTTP 평문 통신임을 가정하고 Web server와 WAS간 통신 모니터링
- 모니터링을 위한 네트워크 환경(탭 스위치)이 구축되어 있다고 가정

일반적인 웹 서비스 구조

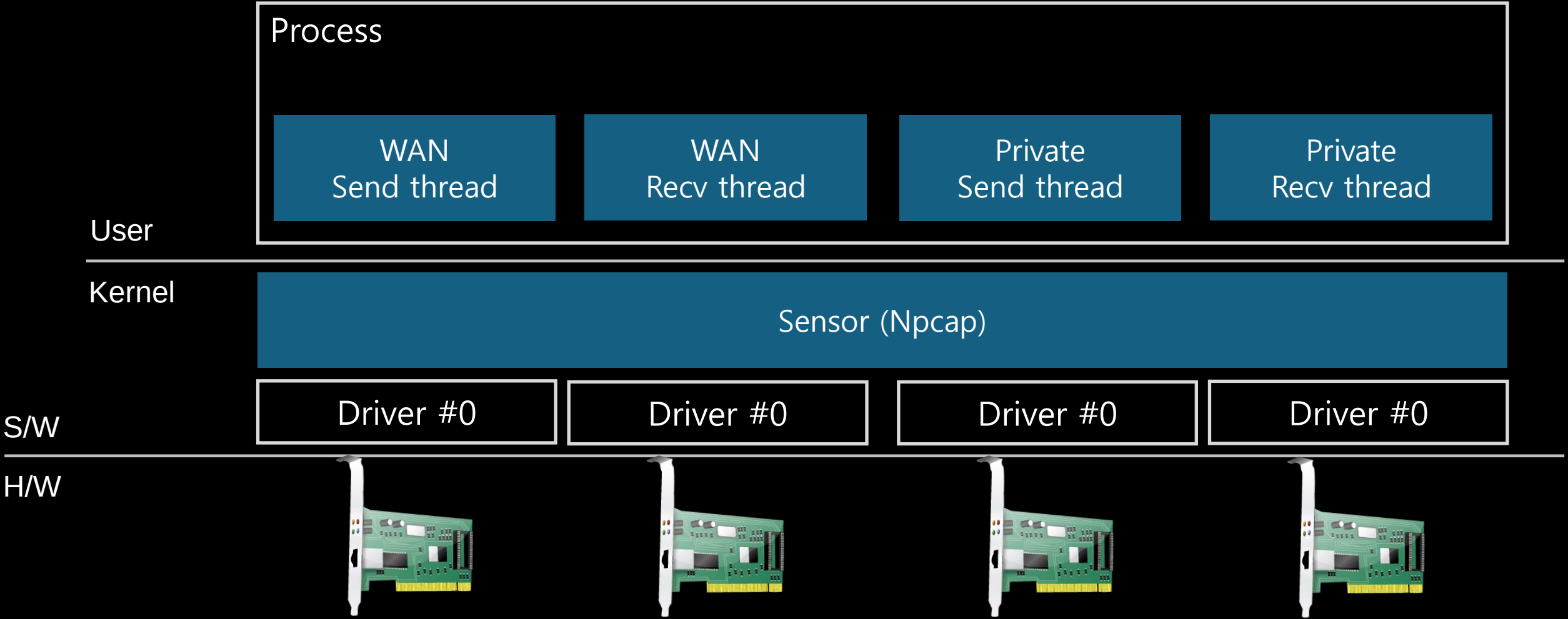


기타 도전과제

- Tab 스위치
 - L2 스위치 기능 포함
 - 포트 미러링
- 패킷 생성 및 전송 소프트웨어
 - GUI 기반 패킷 생성 및 전송
- NAT 기반 인터넷 공유기 직접 제작



인라인 형식 구현



고성능 시스템으로 발전시키기 위한 방법

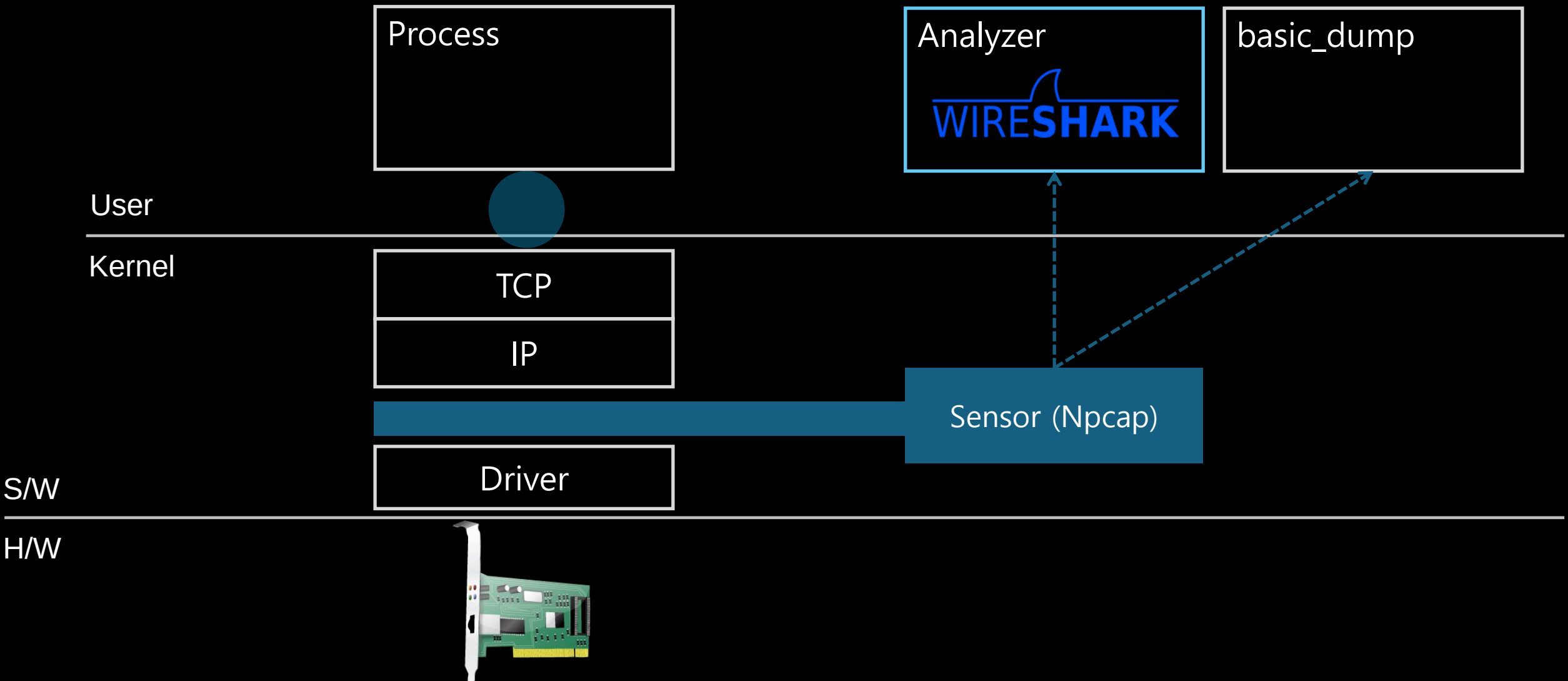
- 지금까지 다룬 예제의 근본적인 문제점

- 패킷 하나를 처리할 때마다 함수가 한 번씩 호출
- 1Gbps 이상의 고성능 시스템에서는 부적합

- 개선 대책

- 한 번에 여러 프레임을 읽고 쓰는 방식으로 개선
- 하드웨어 가속기 활용

예제 작동 구조



감사합니다.